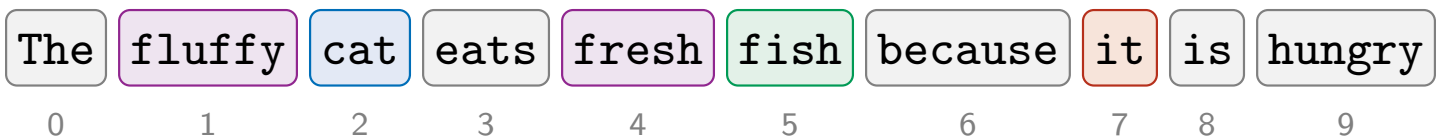
Transformers

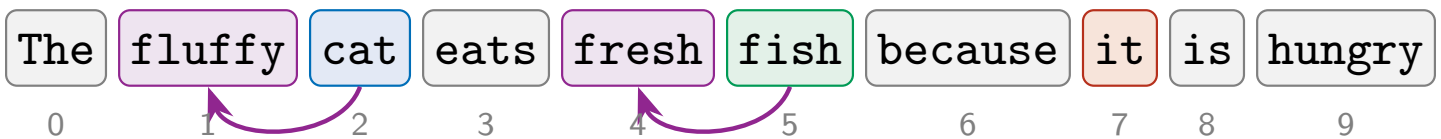
Gersi Doko

Department of Computer Science
University of New Hampshire

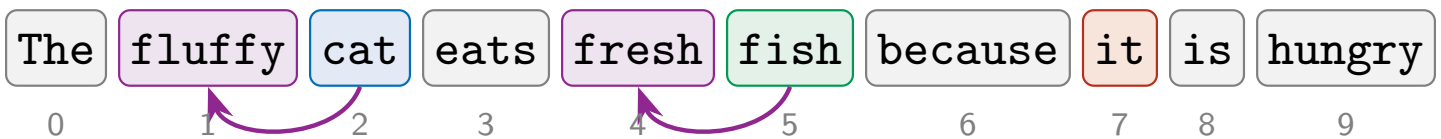
Language is Annoying



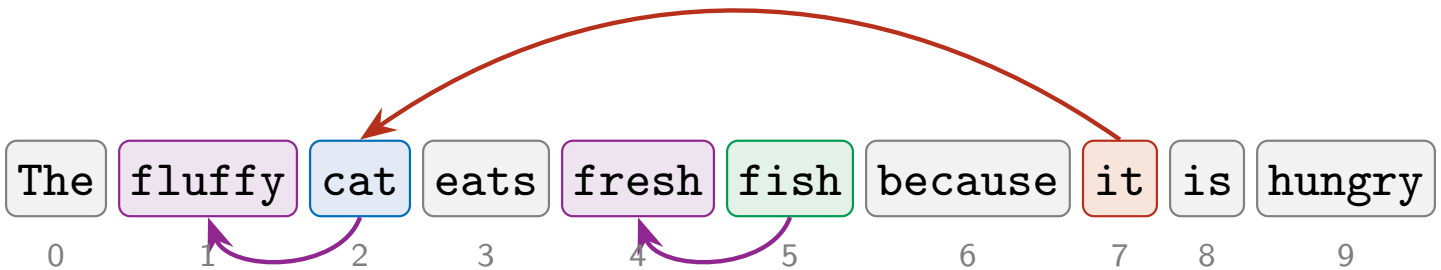
Words Depend on Each Other



Words Depend on Each Other



Words Depend on Each Other



Making a Key Assumption



hungry $\Rightarrow$ it $=$ cat, *tasty* $\Rightarrow$ it $=$ fish

Natural Questions

Q1 What is a word, to a computer?

Q2 Does order matter? - "The fresh fish eats fluffy cat"?

Q3 What does it refer to?

Q4 What is the cat like?

Q5 Can one word take on multiple meanings?

Decoder-Only Architecture



The Big Picture

```
class DecoderOnlyLM(nn.Module):
    ...
    def forward(self, x):
        mask = Batch.subsequent_mask(x.size(1))
        return self.generator(self.blocks(self.embed(x), mask))
```

The Big Picture

```
class DecoderOnlyLM(nn.Module):
    ...
    def forward(self, x):
        mask = Batch.subsequent_mask(x.size(1))
        return self.generator(self.blocks(self.embed(x), mask))
```

1. What are the inputs? - `x (1, 10)`, tokens

The Big Picture

```
class DecoderOnlyLM(nn.Module):
    ...
    def forward(self, x):
        mask = Batch.subsequent_mask(x.size(1))
        return self.generator(self.blocks(self.embed(x), mask))
```

1. What are the inputs? - `x (1, 10)`, tokens
2. What are the outputs? - next token log-probs `(1, 10, V)`

The Big Picture

```
class DecoderOnlyLM(nn.Module):
    ...
    def forward(self, x):
        mask = Batch.subsequent_mask(x.size(1))
        return self.generator(self.blocks(self.embed(x), mask))
```

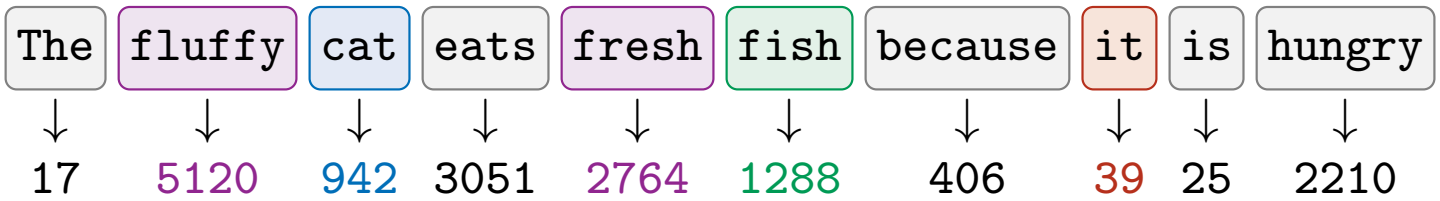
1. What are the inputs? - `x (1, 10)`, tokens
2. What are the outputs? - next token log-probs `(1, 10, V)`
3. "... because it is", how likely is `hungry`? `tasty`?

The Big Picture

```
class DecoderOnlyLM(nn.Module):
    ...
    def forward(self, x):
        mask = Batch.subsequent_mask(x.size(1))
        return self.generator(self.blocks(self.embed(x), mask))
```

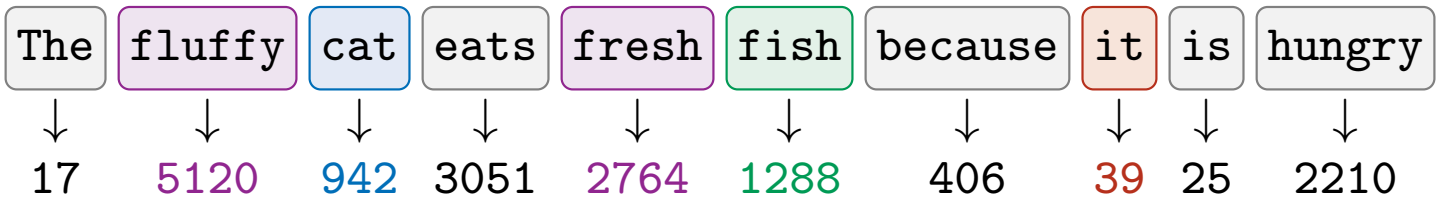
1. What are the inputs? - `x (1, 10)`, tokens
2. What are the outputs? - next token log-probs `(1, 10, V)`
3. "... because it is", how likely is `hungry`? `tasty`?
4. The rest of the talk - what is inside `embed`, `blocks`, `generator`, and why

Word to Vec - 2013



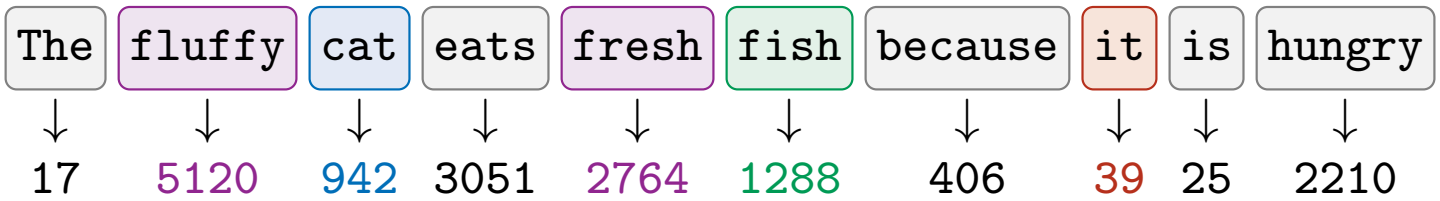
1. An integer - every token in the vocabulary V gets an id

Word to Vec - 2013



1. An integer - every token in the vocabulary V gets an id
2. `tokens` $=$ `[[17, 5120, 942, 3051, 2764, 1288, 406, 39, 25, 2210]]` - shape `(1, 10)`

Word to Vec - 2013

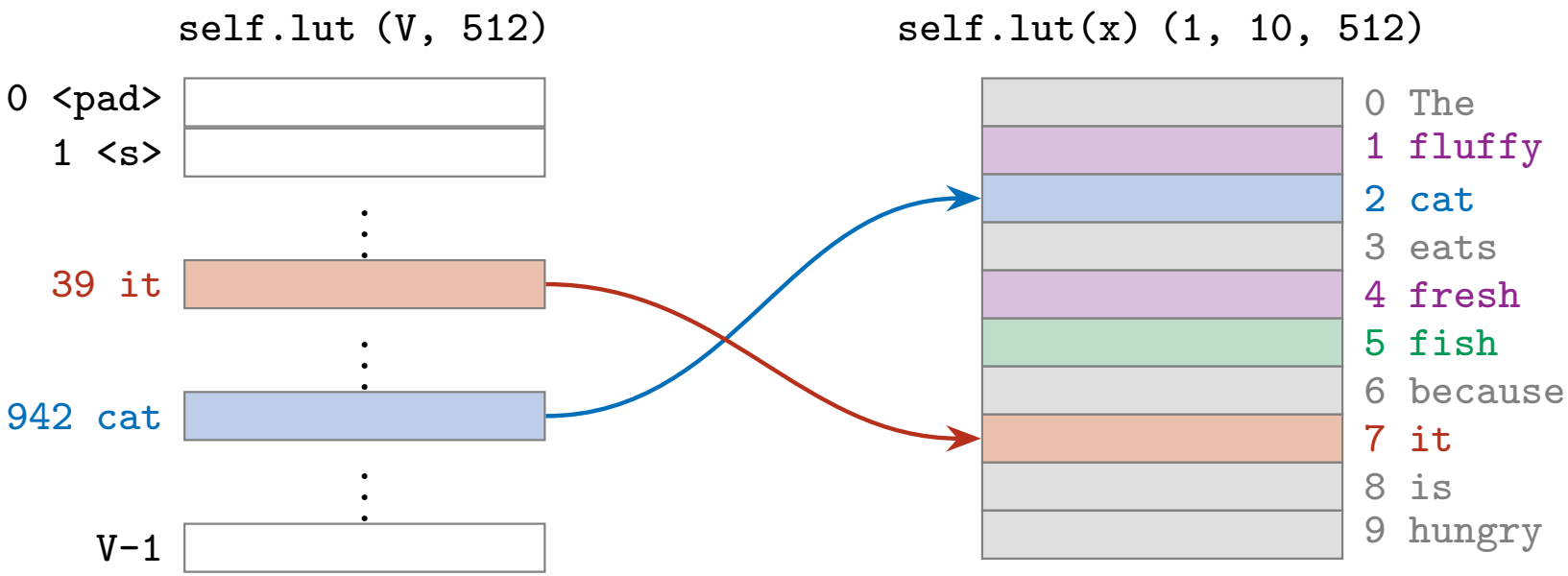


1. An integer - every token in the vocabulary V gets an id
2. `tokens` $=$ `[[17, 5120, 942, 3051, 2764, 1288, 406, 39, 25, 2210]]` - shape `(1, 10)`
3. Some token id's are special 0 $=$ <pad>, 1 $=$ <s>, 2 $=$ </s>, 3 $=$ <unk>, ...

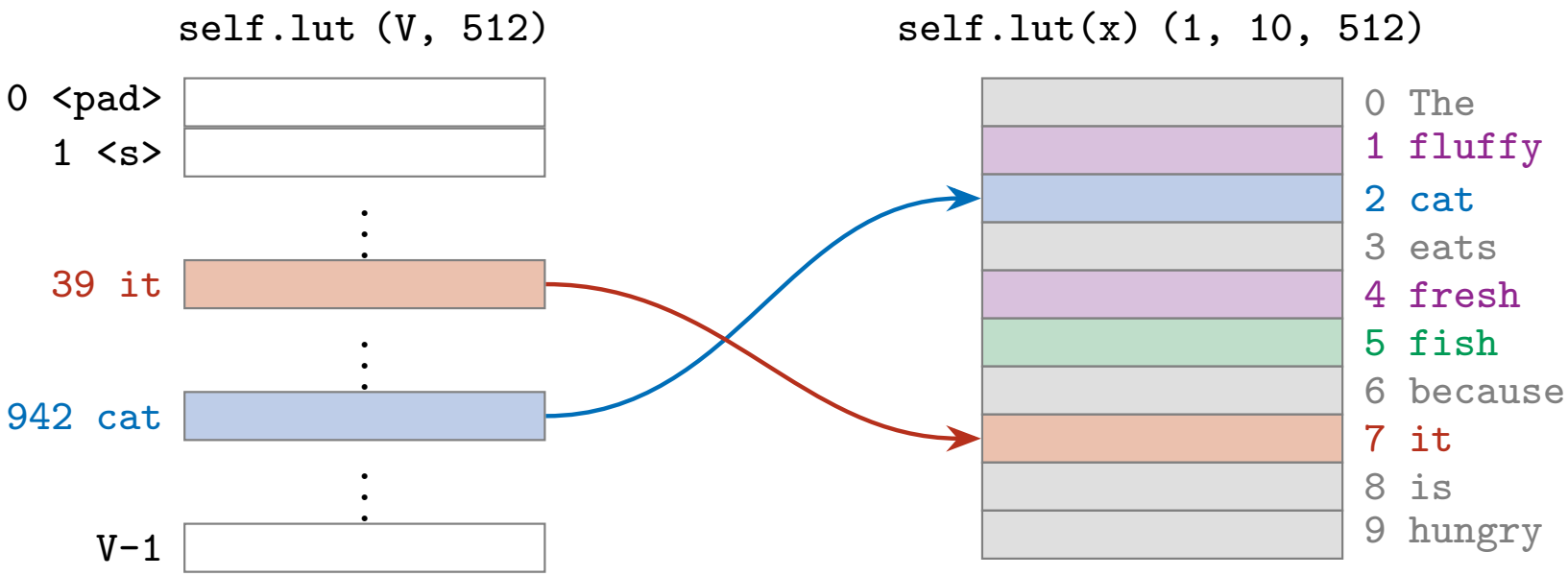
Embeddings

```
class Embeddings(nn.Module):
    def __init__(self, d_model, vocab):
        super(Embeddings, self).__init__()
        self.lut = nn.Embedding(vocab, d_model)
        self.scale_term = math.sqrt(d_model)
    def forward(self, x):
        return self.lut(x) * self.scale_term
```

A (Learnable) Lookup Table

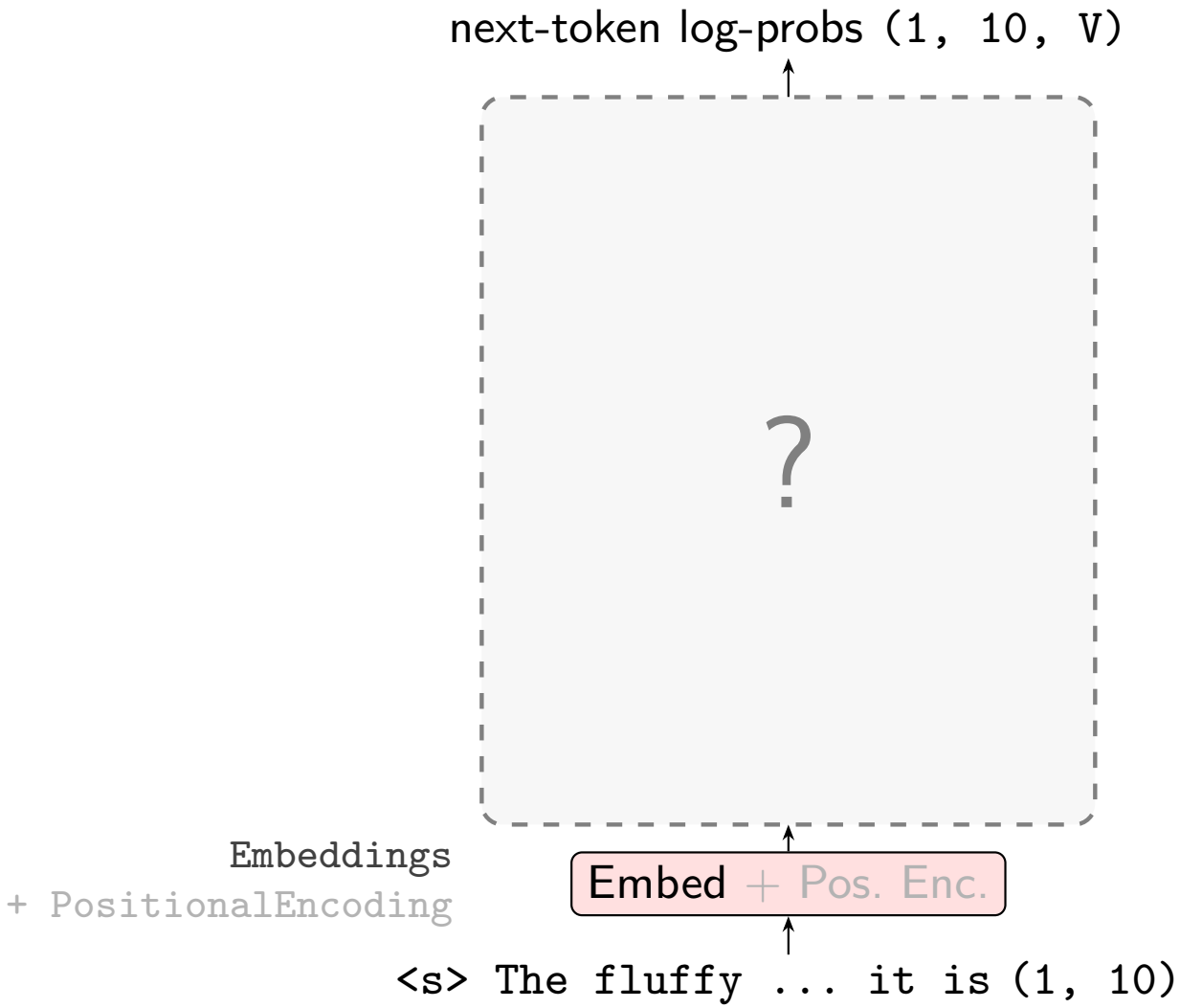


A (Learnable) Lookup Table



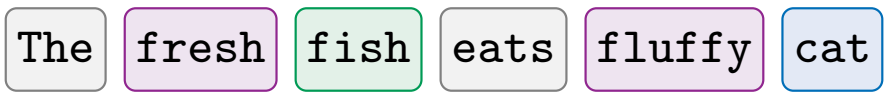
1. Same id ⇒ same row: `it` gets the same vector in every sentence - whether it means the cat or the fish

Decoder-Only Architecture



Order


The
fluffy
cat
eats
fresh
fish


The
fresh
fish
eats
fluffy
cat

Positional Encoding

$$PE_{(pos,2i)} = \sin(pos \cdot \omega_i) \qquad PE_{(pos,2i+1)} = \cos(pos \cdot \omega_i)$$

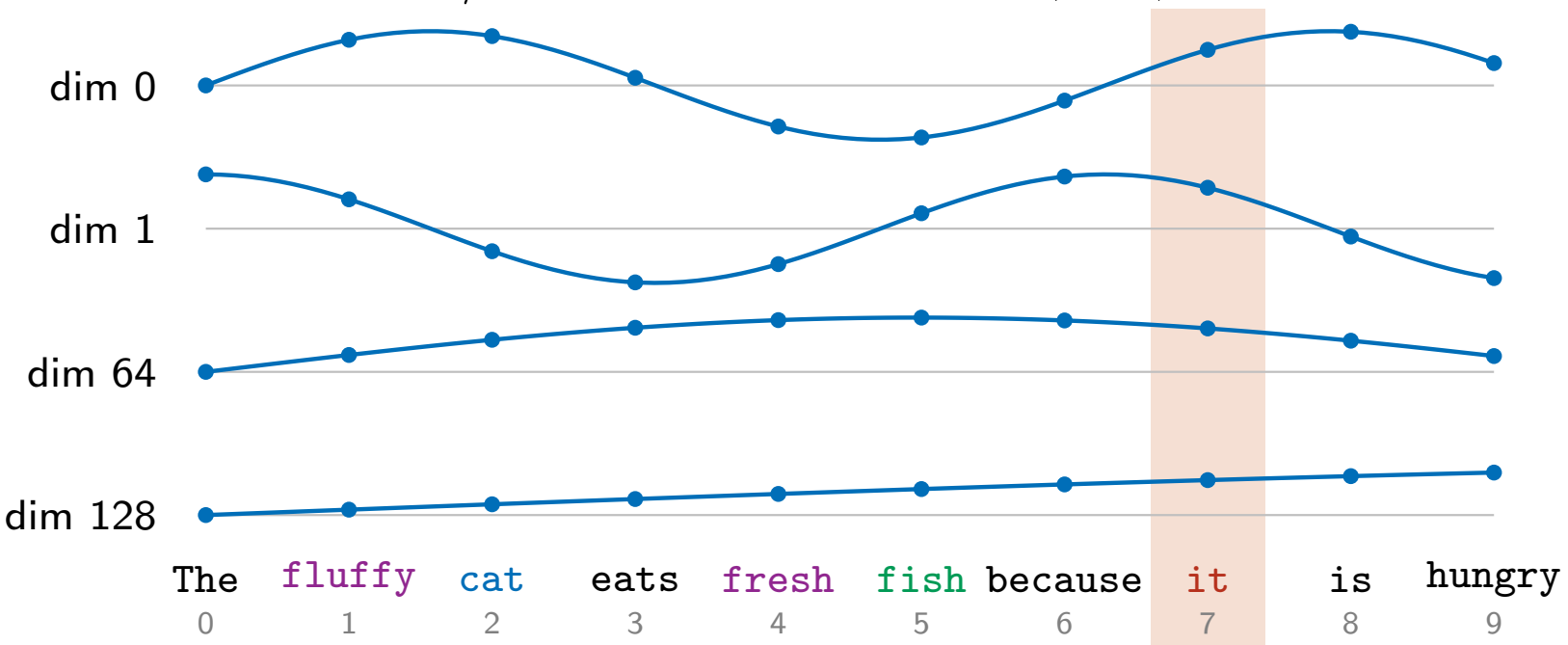
$$\omega_i = 1/10000^{2i/d_{model}}, \quad i = 0, \ldots, 255$$



Positional Encoding

$$PE_{(pos,2i)} = \sin(pos \cdot \omega_i) \qquad PE_{(pos,2i+1)} = \cos(pos \cdot \omega_i)$$

$$\omega_i = 1/10000^{2i/d_{model}}, \quad i = 0, \ldots, 255$$

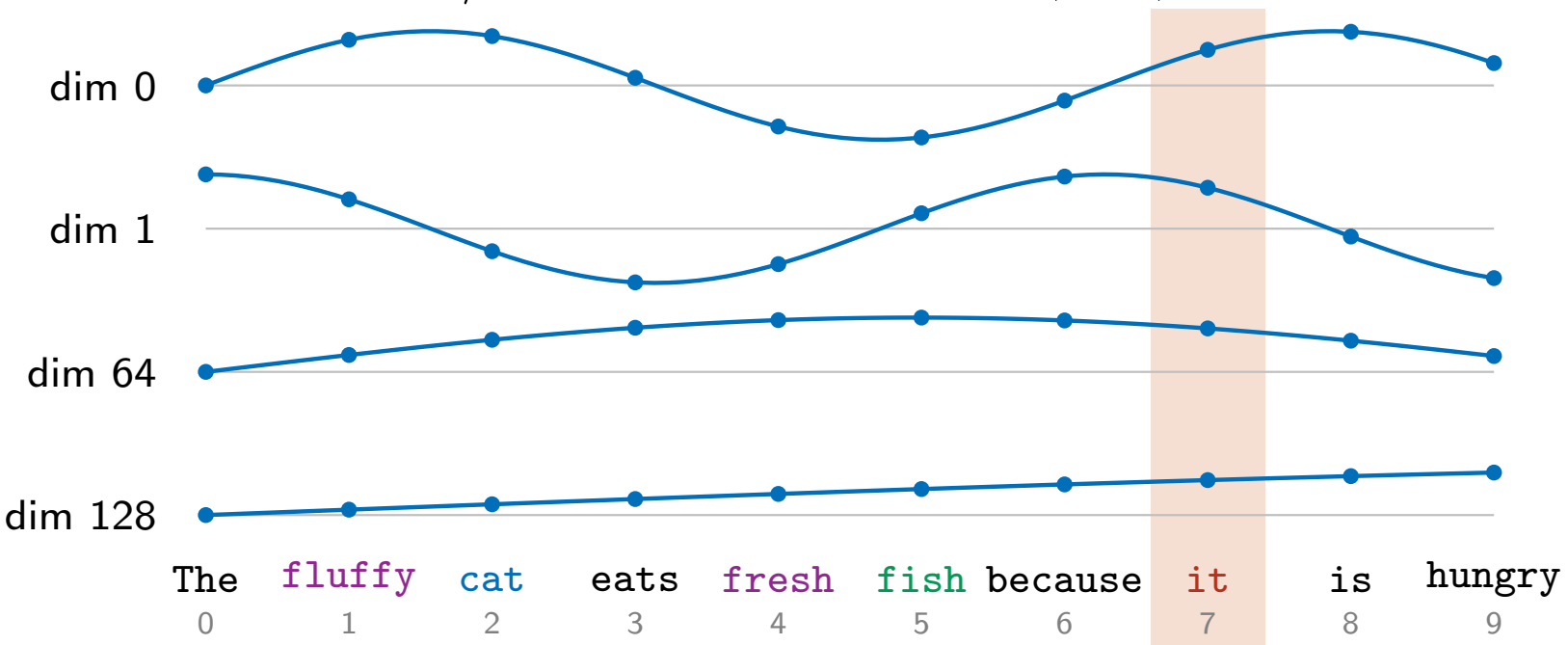


1. The shaded column (4 of its 512 dims) is added to `it` - every position gets its own pattern

Positional Encoding

$$PE_{(pos,2i)} = \sin(pos \cdot \omega_i) \qquad PE_{(pos,2i+1)} = \cos(pos \cdot \omega_i)$$

$$\omega_i = 1/10000^{2i/d_{model}}, \quad i = 0, \ldots, 255$$



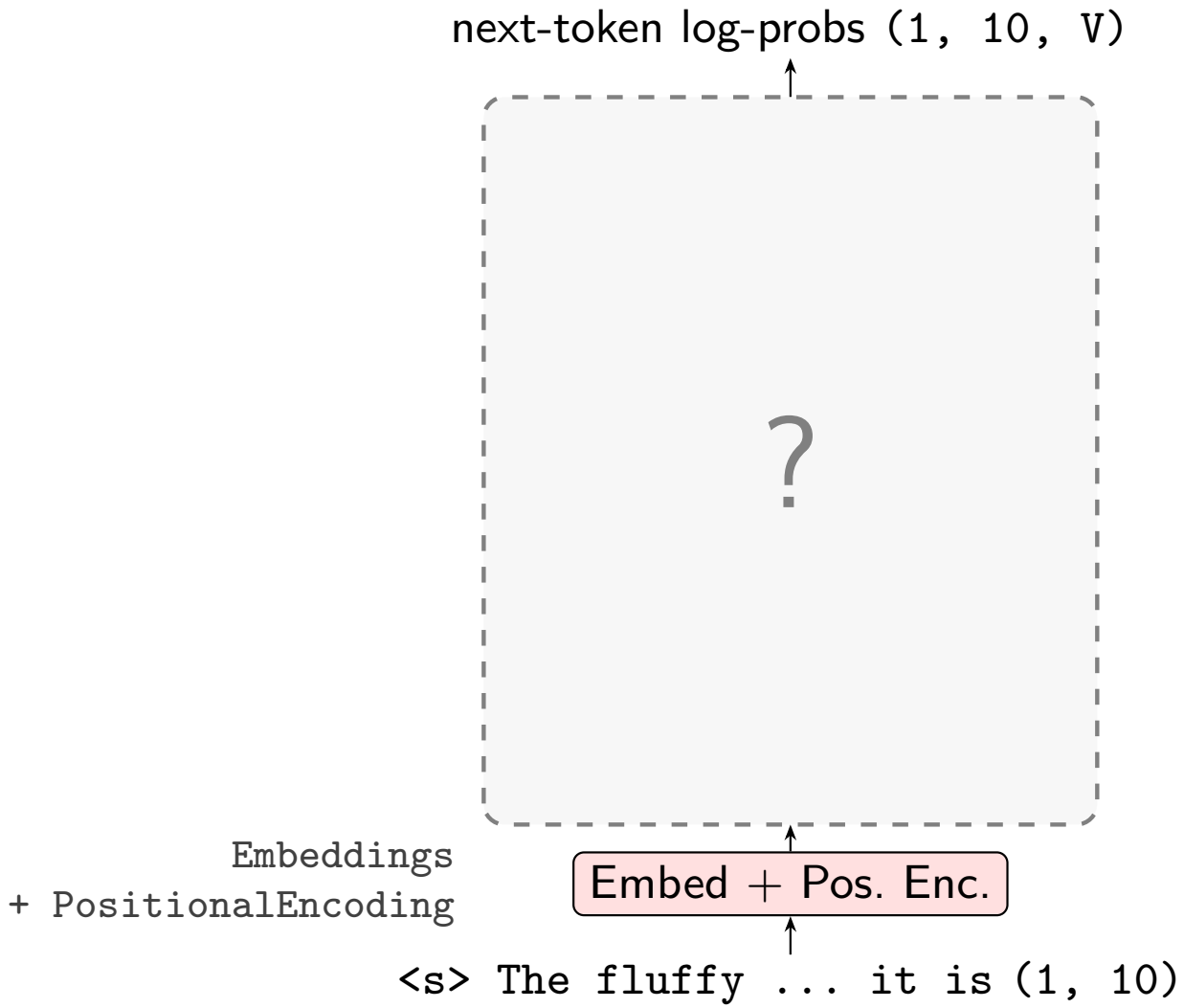
1. The shaded column (4 of its 512 dims) is added to `it` - every position gets its own pattern
2. Fast waves tell neighbors apart, slow waves tell far-apart positions apart

Positional Encoding - The Code

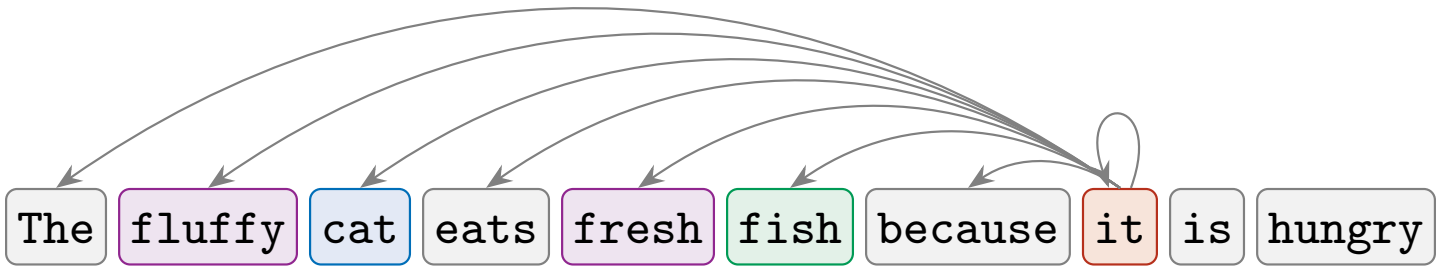
```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, dropout=0.1, max_len=5000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(dropout)
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * -(math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)
    def forward(self, x):
        x = x + Variable(self.pe[:, :x.size(1)], requires_grad=False)
        return self.dropout(x)
```

1. "the word `it`" - knows nothing about `cat` or `fish`

Decoder-Only Architecture

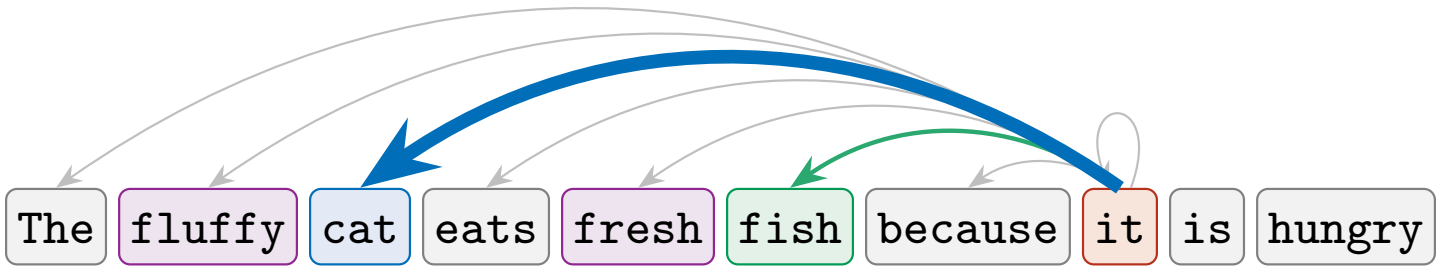


Who Is `it`? Queries, Keys, Values



1. What does `it` refer to?

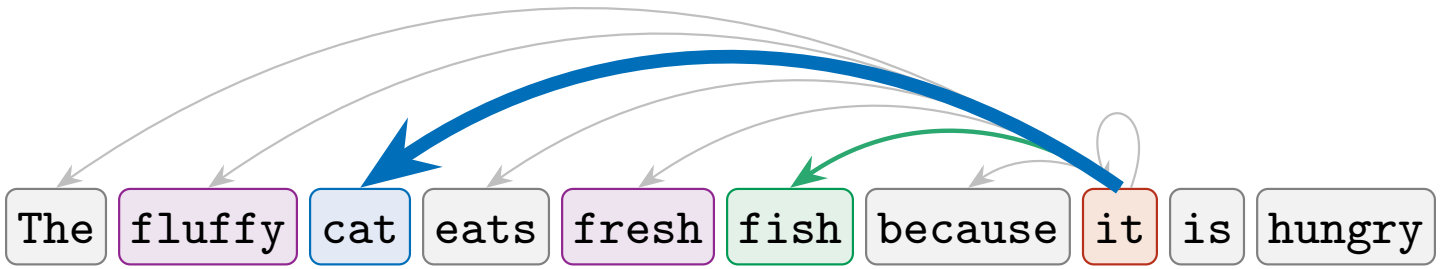
Who Is `it`? Queries, Keys, Values



Illustrative

1. What does `it` refer to?
2. **Query**- what I am looking for?

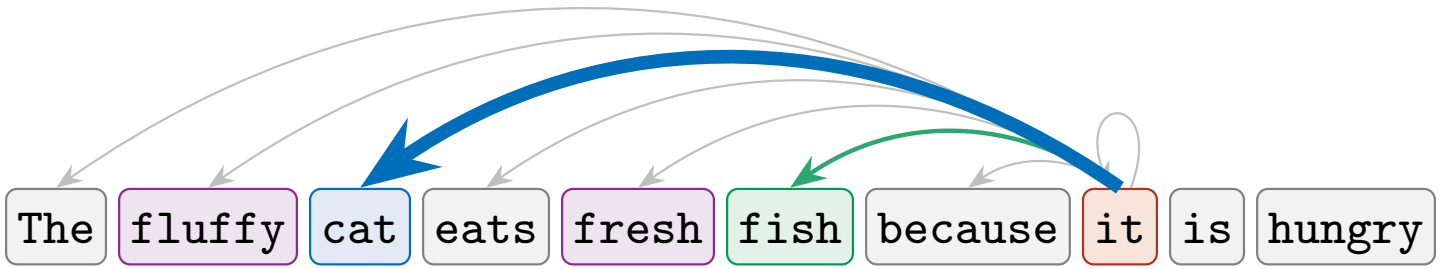
Who Is `it`? Queries, Keys, Values



Illustrative

1. What does `it` refer to?
2. **Query**- what I am looking for?
3. **Key**- what I offer.

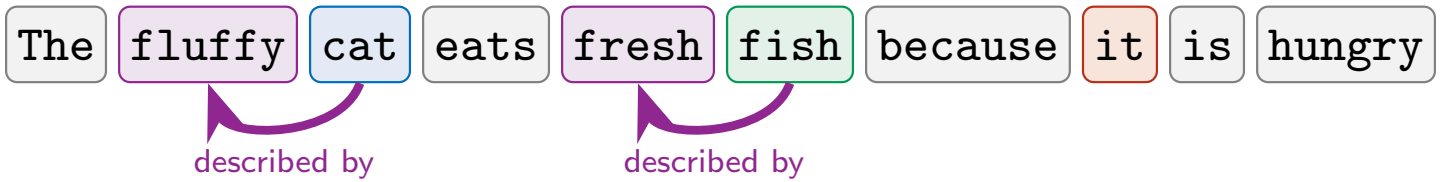
Who Is `it`? Queries, Keys, Values



Illustrative

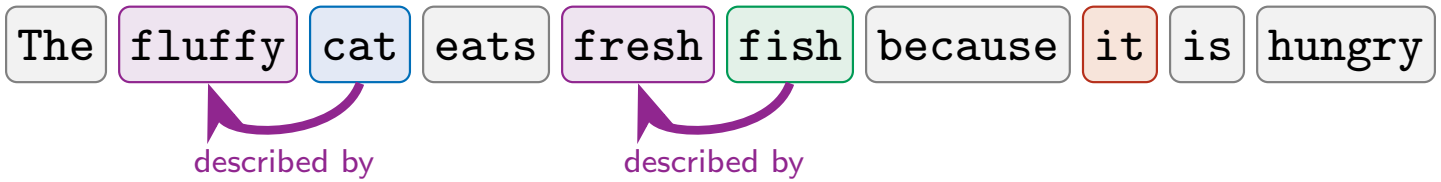
1. What does `it` refer to?
2. **Query**- what I am looking for?
3. **Key**- what I offer.
4. **Value**- what I hand over if I am chosen.

What Is the Cat Like? Another Kind of Query



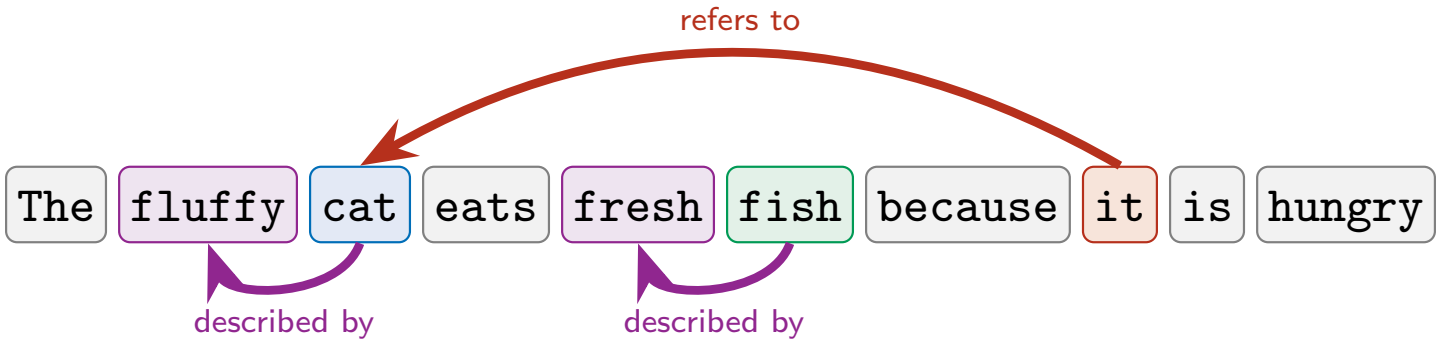
The query asks	Example

What Is the Cat Like? Another Kind of Query



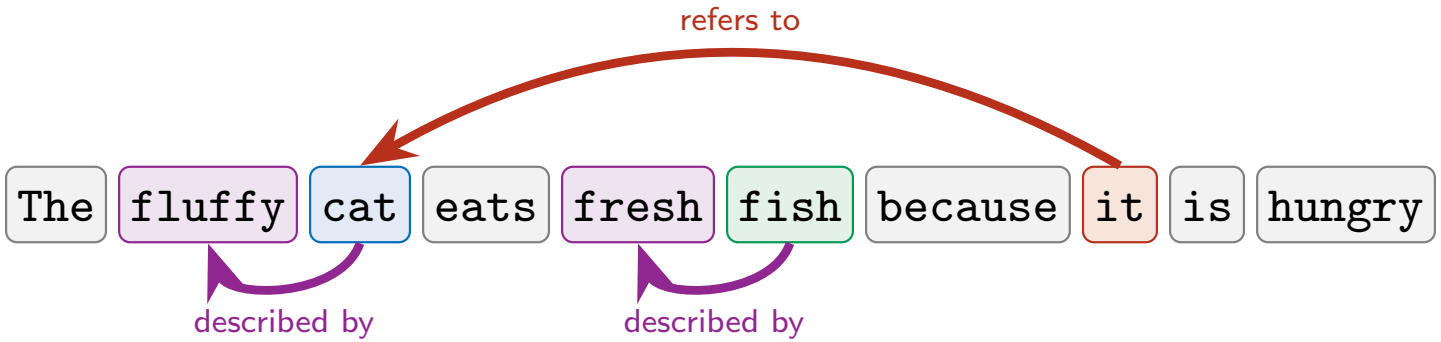
The query asks	Example
"Adj.?"	cat → fluffy fish → fresh

What Is the Cat Like? Another Kind of Query



The query asks	Example
"Adj.?"	cat → fluffy, fish → fresh
"Noun?"	it → cat

What Is the Cat Like? Another Kind of Query



The query asks	Example
"Adj.?"	cat → fluffy, fish → fresh
"Noun?"	it → cat

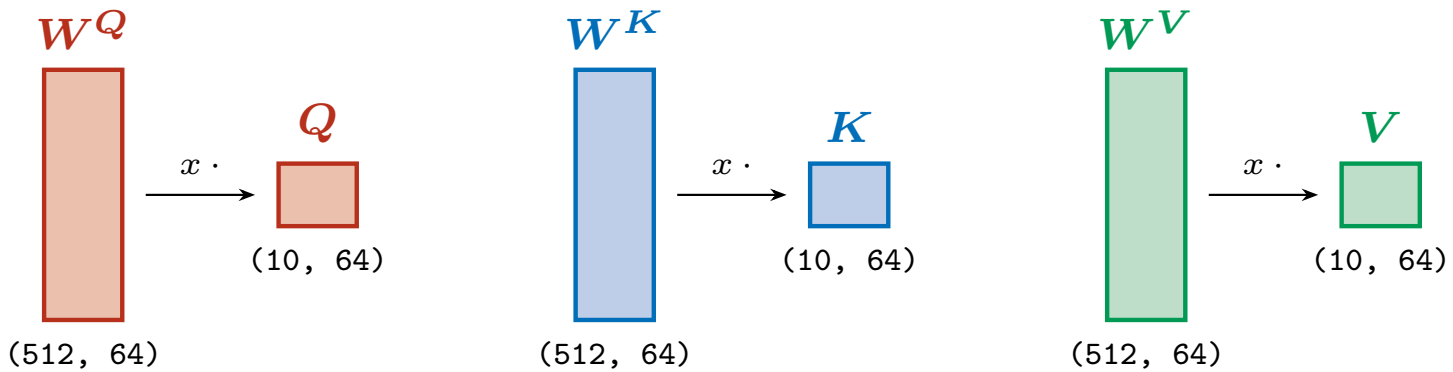
2. Same machinery - only the learned W^Q, W^K differ
3. The value of fluffy is added into cat's vector

$$x_{\texttt{cat}} + \underbrace{0.55\, v_{\texttt{fluffy}} + 0.35\, v_{\texttt{cat}} + 0.10\, v_{\texttt{The}}}_{\text{attention output at cat}} \approx \text{"fluffy cat"}$$

Illustrative weights - the $+$ in front is the residual connection, coming later

Scaled Dot-Product Attention

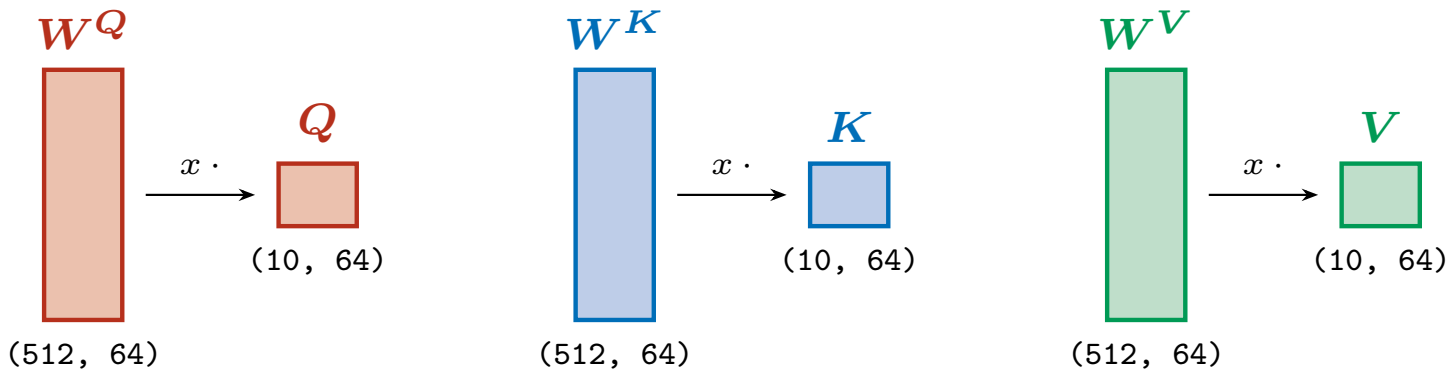
$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}$$



1. $x_i \in \mathbb{R}^{512}$ asks $\boldsymbol{q}_i = x_i\boldsymbol{W}^{\boldsymbol{Q}}$, offers $\boldsymbol{k}_i = x_i\boldsymbol{W}^{\boldsymbol{K}}$, and transmits $\boldsymbol{v}_i = x_i\boldsymbol{W}^{\boldsymbol{V}}$

Scaled Dot-Product Attention

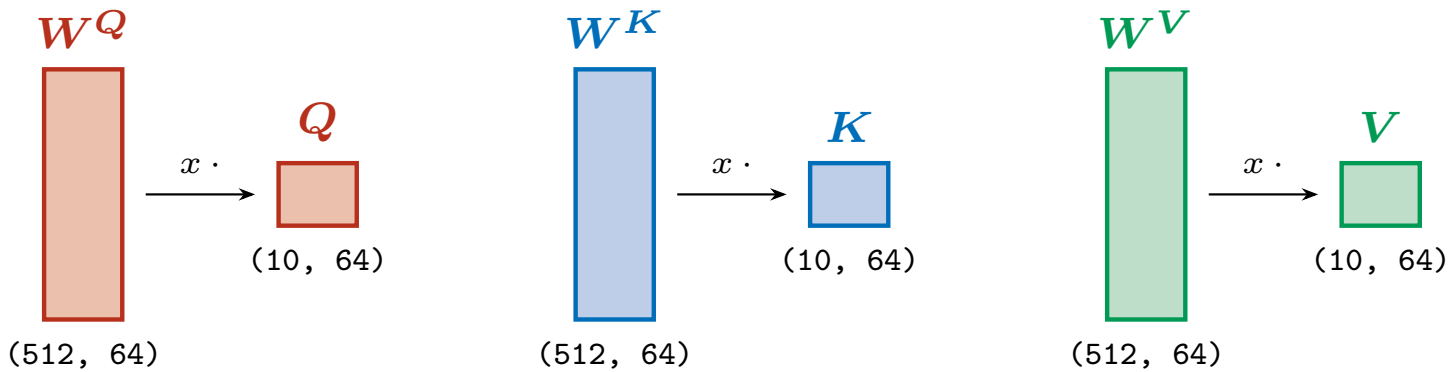
$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}$$



1. $x_i \in \mathbb{R}^{512}$ asks $\boldsymbol{q}_i = x_i\boldsymbol{W}^{\boldsymbol{Q}}$, offers $\boldsymbol{k}_i = x_i\boldsymbol{W}^{\boldsymbol{K}}$, and transmits $\boldsymbol{v}_i = x_i\boldsymbol{W}^{\boldsymbol{V}}$
2. What does $\boldsymbol{q}_i \cdot \boldsymbol{k}_j$ mean?

Scaled Dot-Product Attention

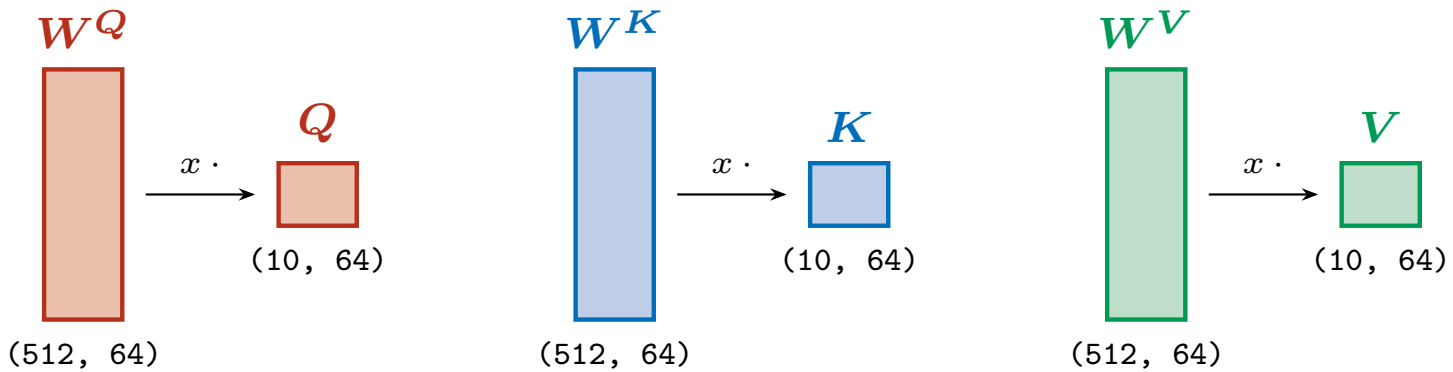
$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}$$



1. $x_i \in \mathbb{R}^{512}$ asks $\boldsymbol{q}_i = x_i\boldsymbol{W}^{\boldsymbol{Q}}$, offers $\boldsymbol{k}_i = x_i\boldsymbol{W}^{\boldsymbol{K}}$, and transmits $\boldsymbol{v}_i = x_i\boldsymbol{W}^{\boldsymbol{V}}$
2. What does $\boldsymbol{q}_i \cdot \boldsymbol{k}_j$ mean?
3. Softmax produces a probability dist. called a 'score'

Scaled Dot-Product Attention

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}$$



1. What are the inputs? - $\boldsymbol{Q}$, $\boldsymbol{K}$, $\boldsymbol{V}$, each (10, 64)
2. What are the outputs? - "Update vector" per token.

The Code: attention

```
def attention(self, Q:torch.Tensor, K:torch.Tensor, V:torch.Tensor, mask = None, dropout = None):
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)
    p_attn = F.softmax(scores, dim = -1)
    if dropout is not None:
        p_attn = dropout(p_attn)
    return torch.matmul(p_attn, V), p_attn
```

Why Divide by $\sqrt{d_k}$?

Why Divide by $\sqrt{d_k}$?

1. $\boldsymbol{q}$ and $\boldsymbol{k}$ are independent with mean 0 and variance 1

Why Divide by $\sqrt{d_k}$?

1. $\boldsymbol{q}$ and $\boldsymbol{k}$ are independent with mean 0 and variance 1
2. $\boldsymbol{q} \cdot \boldsymbol{k} = \sum_{i=1}^{d_k} q_i k_i \sim \mathcal{N}(\mu = 0, \sigma = 8)$

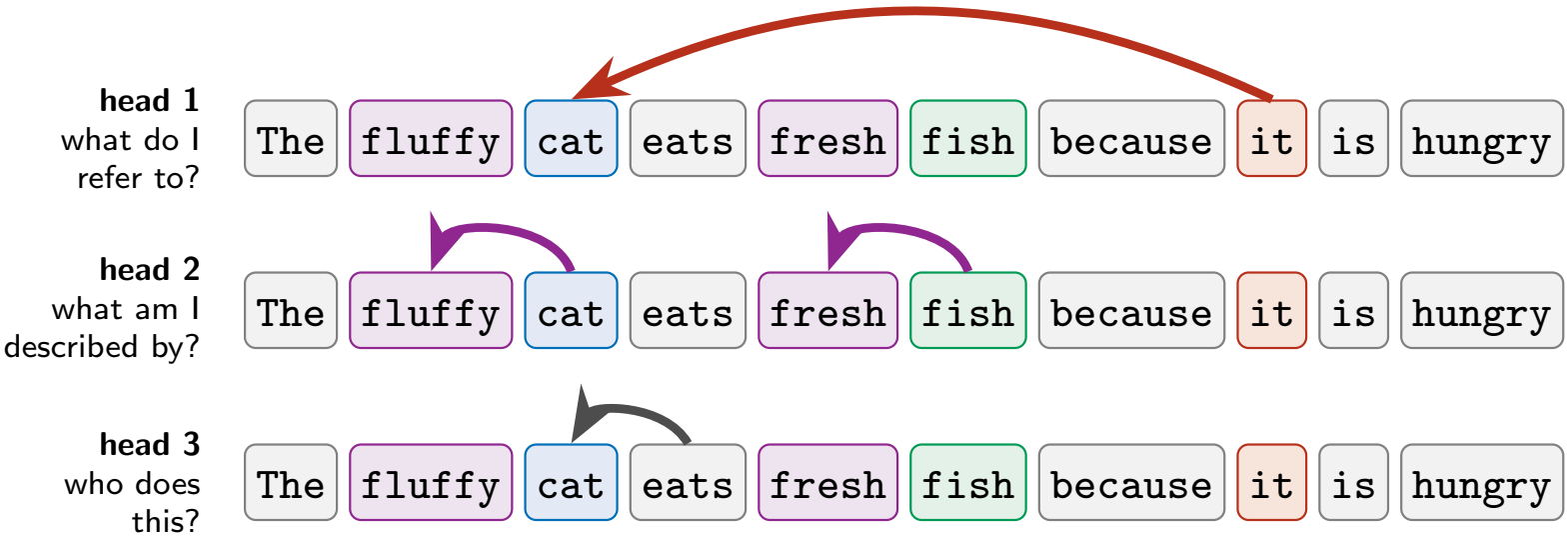
Why Divide by $\sqrt{d_k}$?

1. $\boldsymbol{q}$ and $\boldsymbol{k}$ are independent with mean 0 and variance 1
2. $\boldsymbol{q} \cdot \boldsymbol{k} = \sum_{i=1}^{d_k} q_i k_i \sim \mathcal{N}(\mu = 0, \sigma = 8)$
3. A saturated softmax is almost one-hot - grad vanishes - the model learns slowly
   - $\text{softmax}(1, 2, 3) = (0.09, 0.24, 0.67)$
   - $\text{softmax}(8, 16, 24) \approx (0.00, 0.00, 1.00)$

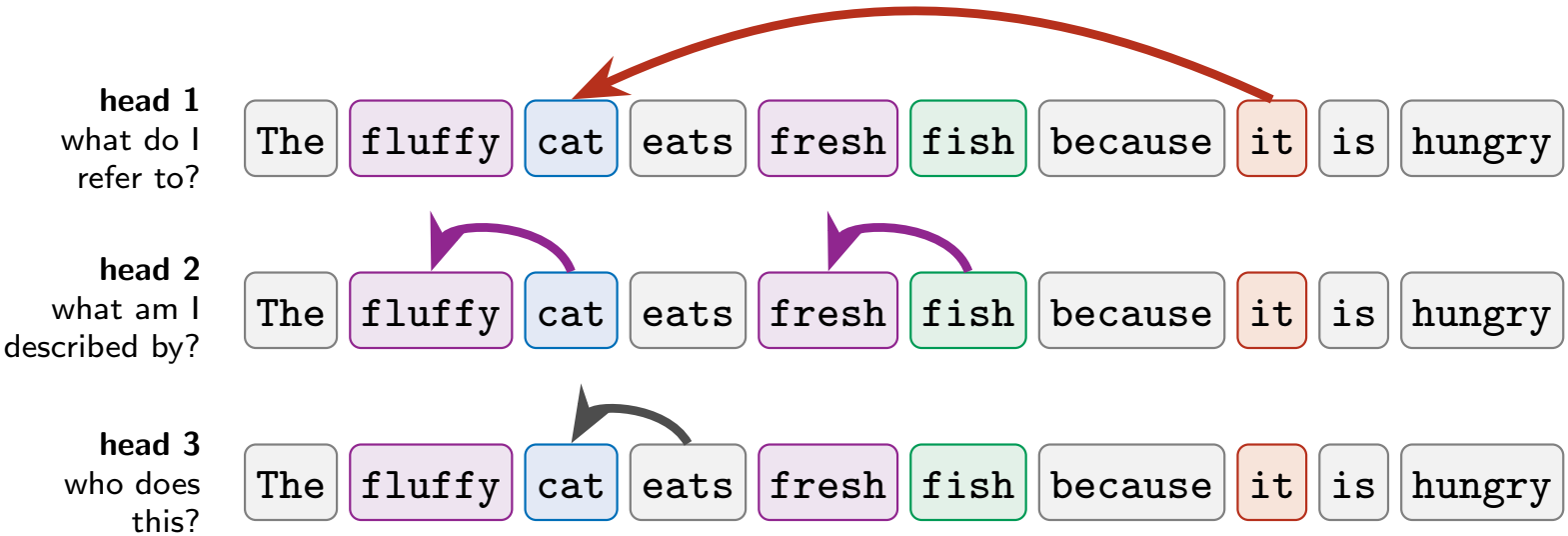
Why Divide by $\sqrt{d_k}$?

1. $\boldsymbol{q}$ and $\boldsymbol{k}$ are independent with mean 0 and variance 1
2. $\boldsymbol{q} \cdot \boldsymbol{k} = \sum_{i=1}^{d_k} q_i k_i \sim \mathcal{N}(\mu = 0, \sigma = 8)$
3. A saturated softmax is almost one-hot - grad vanishes - the model learns slowly
   - $\mathrm{softmax}(1, 2, 3) = (0.09, 0.24, 0.67)$
   - $\mathrm{softmax}(8, 16, 24) \approx (0.00, 0.00, 1.00)$
4. Divide by $\sqrt{d_k} = 8$ - the scores have variance 1 again

Many Questions at Once

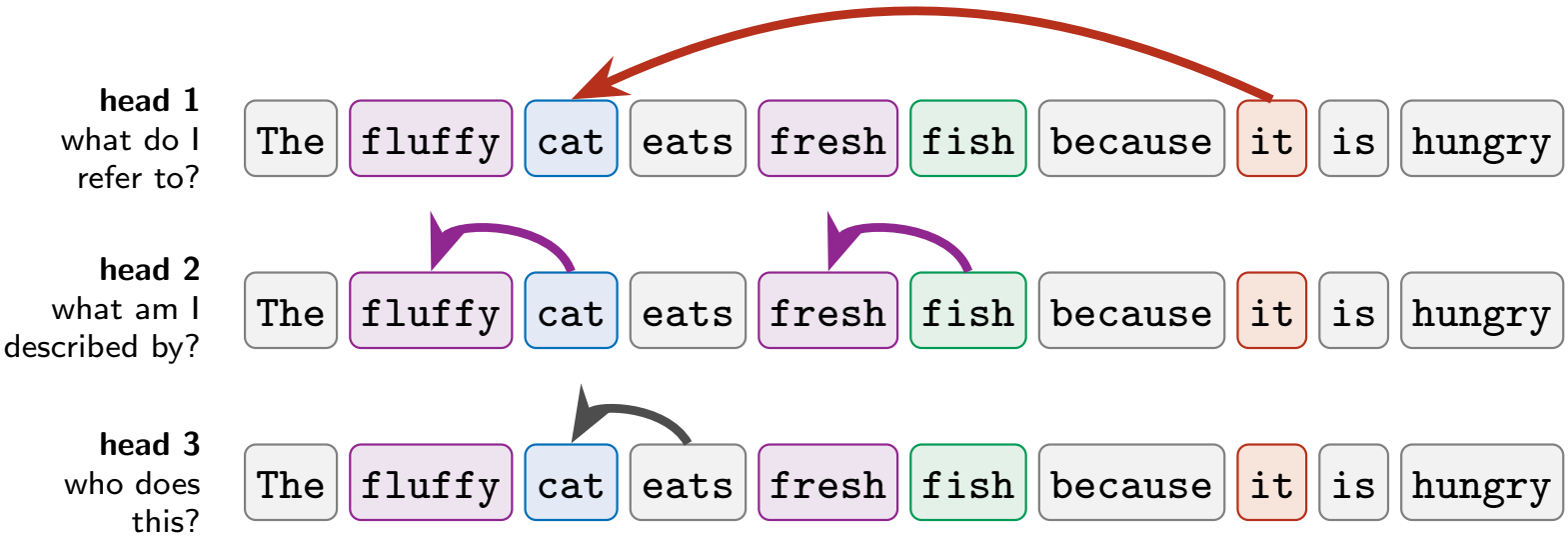


Many Questions at Once



1. $h = 3$ heads, each with its own $\boldsymbol{W}_j^{\boldsymbol{Q}}, \boldsymbol{W}_j^{\boldsymbol{K}}, \boldsymbol{W}_j^{\boldsymbol{V}} \in \mathbb{R}^{512\times64}$

Many Questions at Once



1. $h = 3$ heads, each with its own $\boldsymbol{W}_j^{\boldsymbol{Q}}, \boldsymbol{W}_j^{\boldsymbol{K}}, \boldsymbol{W}_j^{\boldsymbol{V}} \in \mathbb{R}^{512\times64}$
2. The 3 "update mats" are sent to FC Layer $W^O \in \mathbb{R}^{512\times512}$ then a Residual Connection

The Code: MultiHeadedAttention

```
class MultiHeadedAttention(nn.Module):
  def __init__(self, h:int, d_model:int, dropout:float=0.1):
      ...
      self.d_k = d_model // h
      self.h = h
      # every head j gets its own W_q[j], W_k[j], W_v[j]: 512 -> 64
      self.W_q = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_k = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_v = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_o = nn.Linear(d_model, d_model)
      ...
  def forward(self, x, mask=None):
      heads = []
      for j in range(self.h):
          Q, K, V = self.W_q[j](x), self.W_k[j](x), self.W_v[j](x)  # (1, 10, 64) each
          out_j, _ = self.attention(Q, K, V, mask=mask, dropout=self.dropout)  # (1, 10, 64)
          heads.append(out_j)
      return self.W_o(torch.cat(heads, dim=-1))  # 8 x (1, 10, 64) -> (1, 10, 512)
```

The Code: `MultiHeadedAttention`

```
class MultiHeadedAttention(nn.Module):
  def __init__(self, h:int, d_model:int, dropout:float=0.1):
      ...
      self.d_k = d_model // h
      self.h = h
      # every head j gets its own W_q[j], W_k[j], W_v[j]: 512 -> 64
      self.W_q = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_k = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_v = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_o = nn.Linear(d_model, d_model)
      ...
  def forward(self, x, mask=None):
      heads = []
      for j in range(self.h):
          Q, K, V = self.W_q[j](x), self.W_k[j](x), self.W_v[j](x)  # (1, 10, 64) each
          out_j, _ = self.attention(Q, K, V, mask=mask, dropout=self.dropout)  # (1, 10, 64)
          heads.append(out_j)
      return self.W_o(torch.cat(heads, dim=-1))  # 8 x (1, 10, 64) -> (1, 10, 512)
```

1. What are the inputs? - `x (1, 10, 512)` - Q, K, V all come from the same `x` (*self*-attention)

The Code: `MultiHeadedAttention`

```
class MultiHeadedAttention(nn.Module):
  def __init__(self, h:int, d_model:int, dropout:float=0.1):
      ...
      self.d_k = d_model // h
      self.h = h
      # every head j gets its own W_q[j], W_k[j], W_v[j]: 512 -> 64
      self.W_q = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_k = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_v = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_o = nn.Linear(d_model, d_model)
      ...
  def forward(self, x, mask=None):
      heads = []
      for j in range(self.h):
          Q, K, V = self.W_q[j](x), self.W_k[j](x), self.W_v[j](x)  # (1, 10, 64) each
          out_j, _ = self.attention(Q, K, V, mask=mask, dropout=self.dropout)  # (1, 10, 64)
          heads.append(out_j)
      return self.W_o(torch.cat(heads, dim=-1))  # 8 x (1, 10, 64) -> (1, 10, 512)
```

1. What are the inputs? - `x (1, 10, 512)` - Q, K, V all come from the same `x` (*self*-attention)
2. What are the outputs? - `(1, 10, 512)` - the "update vector" for each token

The Code: MultiHeadedAttention

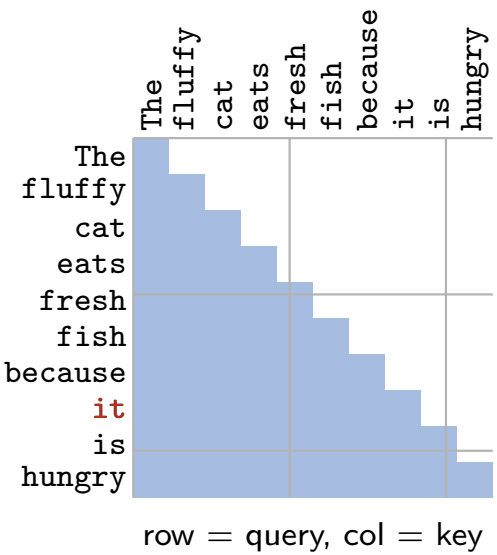
```
class MultiHeadedAttention(nn.Module):
  def __init__(self, h:int, d_model:int, dropout:float=0.1):
      ...
      self.d_k = d_model // h
      self.h = h
      # every head j gets its own W_q[j], W_k[j], W_v[j]: 512 -> 64
      self.W_q = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_k = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_v = nn.ModuleList([nn.Linear(d_model, self.d_k) for _ in range(h)])
      self.W_o = nn.Linear(d_model, d_model)
      ...
  def forward(self, x, mask=None):
      heads = []
      for j in range(self.h):
          Q, K, V = self.W_q[j](x), self.W_k[j](x), self.W_v[j](x)  # (1, 10, 64) each
          out_j, _ = self.attention(Q, K, V, mask=mask, dropout=self.dropout)  # (1, 10, 64)
          heads.append(out_j)
      return self.W_o(torch.cat(heads, dim=-1))  # 8 x (1, 10, 64) -> (1, 10, 512)
```

1. What are the inputs? - `x (1, 10, 512)` - $\boldsymbol{Q}$, $\boldsymbol{K}$, $\boldsymbol{V}$ all come from the same `x` (*self*-attention)
2. What are the outputs? - `(1, 10, 512)` - the "update vector" for each token
3. Each head j has its own $\boldsymbol{W}_j^{\boldsymbol{Q}}, \boldsymbol{W}_j^{\boldsymbol{K}}, \boldsymbol{W}_j^{\boldsymbol{V}} \in \mathbb{R}^{512\times64}$, plus one shared $W^O \in \mathbb{R}^{512\times512}$

No Peeking - subsequent_mask

```
def subsequent_mask(size):
    attn_shape = (1, size, size)
    subsequent_mask = np.triu(np.ones(attn_shape), k=1).astype('uint8')
    return torch.from_numpy(subsequent_mask) == 0
```

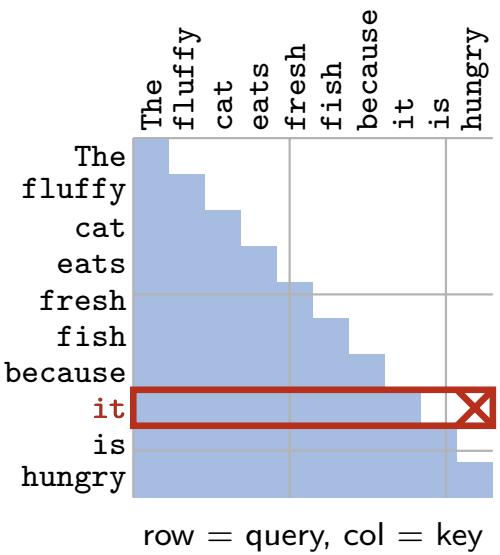
```
if mask is not None:
    scores = scores.masked_fill(mask == 0, -1e9)
p_attn = F.softmax(scores, dim = -1)
```



No Peeking - subsequent_mask

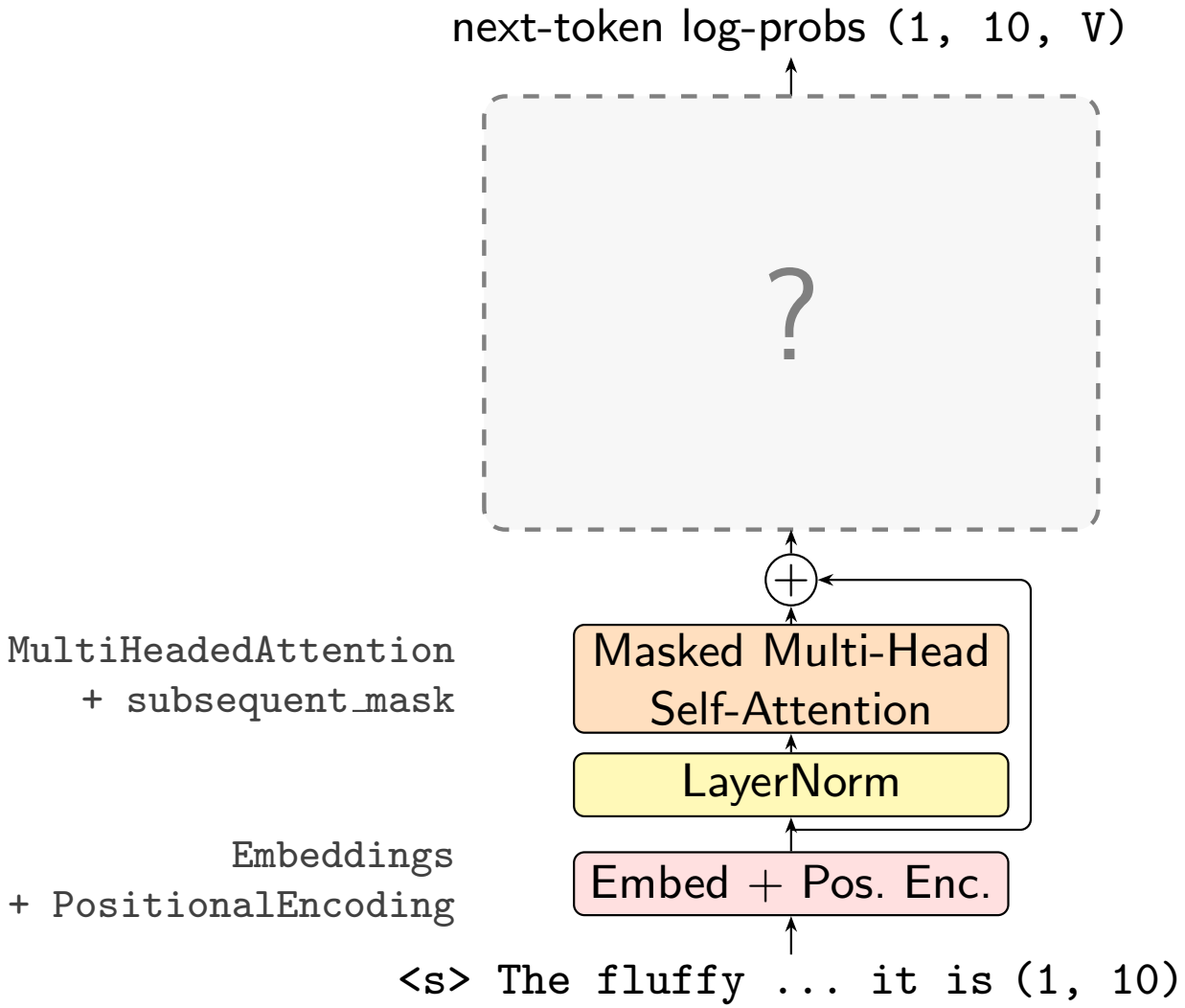
```
def subsequent_mask(size):
    attn_shape = (1, size, size)
    subsequent_mask = np.triu(np.ones(attn_shape), k=1).astype('uint8')
    return torch.from_numpy(subsequent_mask) == 0
```

```
if mask is not None:
    scores = scores.masked_fill(mask == 0, -1e9)
p_attn = F.softmax(scores, dim = -1)
```



1. Row `it` cannot see `is`, `hungry` - when we write, they do not exist yet

Decoder-Only Architecture



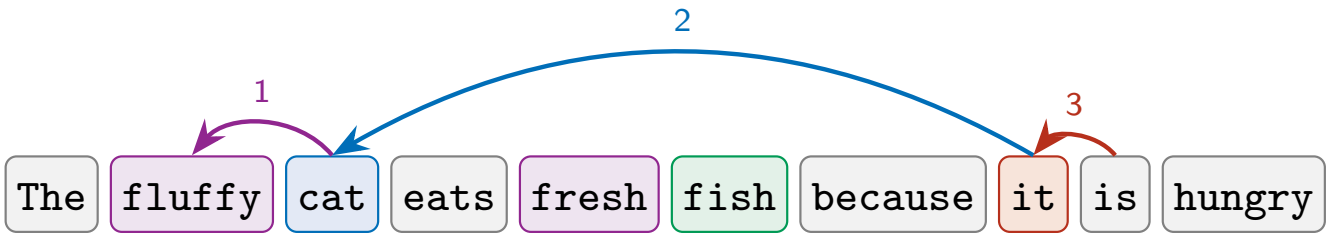
What Do We Know About Cats?

Cats get hungry, fish get eaten - none of that is *in* the sentence

```
class PositionwiseFeedForward(nn.Module):
    def __init__(self, d_model: int, d_ff: int, dropout:float=0.1):
        super(PositionwiseFeedForward, self).__init__()
        self.w1 = nn.Linear(d_model, d_ff)
        self.w2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)
    def forward(self, x:torch.Tensor):
        return self.w2(self.dropout(F.selu(self.w1(x))))
```

Attention mixes *across* tokens, the FFN works *within* a token;
"position-wise" = per position - it adds no position information

Numerical Stability: Residuals and LayerNorm



cat ← fluffy, then it ← cat, then is ← it, plus "cats get hungry" - several rounds ⇒ stack $N = 6$ blocks

```python
class LayerNorm(nn.Module):
    def forward(self, x):
        mean = x.mean(-1, keepdim=True)
        std = x.std(-1, keepdim=True)
        return self.alpha * ((x - mean) / (std + self.eps)) + self.bias
```

```python
class EncoderLayer(nn.Module):
    def forward(self, x, mask):
        x = x + self.dropout(self.self_attn(self.norm1(x), mask))  # attention
        return x + self.dropout(self.feed_forward(self.norm2(x)))  # feed-forward
```

GPT2 - 2019

```
class EncoderLayer(nn.Module):
    def __init__(self, size, self_attn, feed_forward, dropout):
        ...
        self.norm1, self.norm2 = LayerNorm(size), LayerNorm(size)
    def forward(self, x, mask):
        x = x + self.dropout(self.self_attn(self.norm1(x), mask))
        return x + self.dropout(self.feed_forward(self.norm2(x)))
```

```
class Encoder(nn.Module):
    def __init__(self, layer, N):
        ...
        self.layers = nn.ModuleList([copy.deepcopy(layer) for _ in range(N)])
        self.norm = LayerNorm(layer.size)
    def forward(self, x, mask):
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)
```

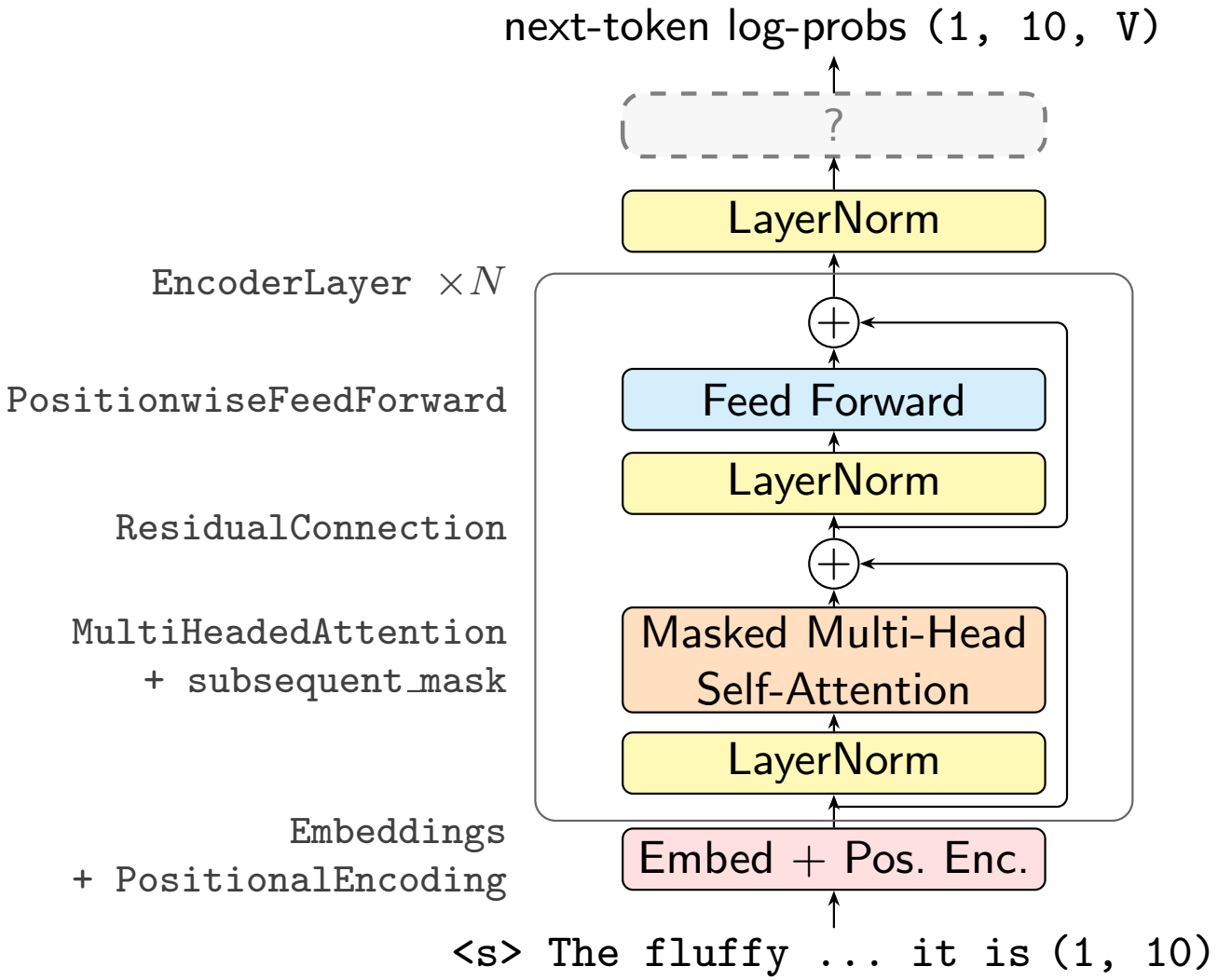
GPT2 - 2019

```
class EncoderLayer(nn.Module):
    def __init__(self, size, self_attn, feed_forward, dropout):
        ...
        self.norm1, self.norm2 = LayerNorm(size), LayerNorm(size)
    def forward(self, x, mask):
        x = x + self.dropout(self.self_attn(self.norm1(x), mask))
        return x + self.dropout(self.feed_forward(self.norm2(x)))
```

```
class Encoder(nn.Module):
    def __init__(self, layer, N):
        ...
        self.layers = nn.ModuleList([copy.deepcopy(layer) for _ in range(N)])
        self.norm = LayerNorm(layer.size)
    def forward(self, x, mask):
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)
```

1. What are the inputs? - `x (1, 10, 512)`, `mask (1, 10, 10)`
2. What are the outputs? - `(1, 10, 512)` - the enhanced representation of each token, after N rounds

Decoder-Only Architecture



Choosing the Next Word - Generator

```
class Generator(nn.Module):
    def __init__(self, d_model, vocab):
        super(Generator, self).__init__()
        self.proj = nn.Linear(d_model, vocab)
    def forward(self, x):
        return F.log_softmax(self.proj(x), dim=-1)
```

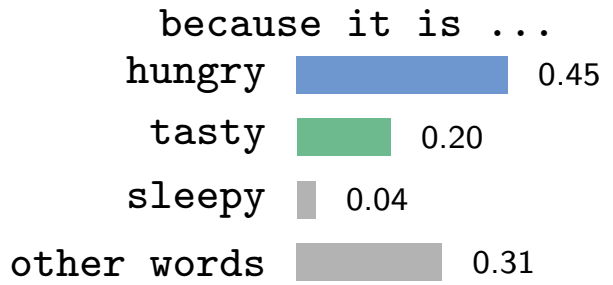
Choosing the Next Word - `Generator`

```
class Generator(nn.Module):
    def __init__(self, d_model, vocab):
        super(Generator, self).__init__()
        self.proj = nn.Linear(d_model, vocab)
    def forward(self, x):
        return F.log_softmax(self.proj(x), dim=-1)
```

1. What are the inputs? - enhanced token representations `(1, 10, 512)`

Choosing the Next Word - Generator

```
class Generator(nn.Module):
    def __init__(self, d_model, vocab):
        super(Generator, self).__init__()
        self.proj = nn.Linear(d_model, vocab)
    def forward(self, x):
        return F.log_softmax(self.proj(x), dim=-1)
```



1. What are the inputs? - enhanced token representations `(1, 10, 512)`
2. What are the outputs? - `(1, 10, V)` - log-probs for the word after position t

Our GPT2 - 2019

```
class DecoderOnlyLM(nn.Module):
    def __init__(self, V, N=6, d_model=512, d_ff=1024, h=8, dropout=0.1):
        super().__init__()
        c = copy.deepcopy
        attn, ff = MultiHeadedAttention(h, d_model), PositionwiseFeedForward(d_model, d_ff, dropout)
        self.embed = nn.Sequential(Embeddings(d_model, V), PositionalEncoding(d_model, dropout))
        self.blocks = Encoder(EncoderLayer(d_model, c(attn), c(ff), dropout), N)
        self.generator = Generator(d_model, V)
        [nn.init.xavier_uniform_(p) for p in self.parameters() if p.dim() > 1]
    def forward(self, x):                                # x: (batch, n) token ids
        mask = Batch.subsequent_mask(x.size(1))          # (1, n, n), True on/below diagonal
        return self.generator(self.blocks(self.embed(x), mask))   # (batch, n, V) log-probs
```

Our GPT2 - 2019

```
class DecoderOnlyLM(nn.Module):
    def __init__(self, V, N=6, d_model=512, d_ff=1024, h=8, dropout=0.1):
        super().__init__()
        c = copy.deepcopy
        attn, ff = MultiHeadedAttention(h, d_model), PositionwiseFeedForward(d_model, d_ff, dropout)
        self.embed = nn.Sequential(Embeddings(d_model, V), PositionalEncoding(d_model, dropout))
        self.blocks = Encoder(EncoderLayer(d_model, c(attn), c(ff), dropout), N)
        self.generator = Generator(d_model, V)
        [nn.init.xavier_uniform_(p) for p in self.parameters() if p.dim() > 1]
    def forward(self, x):                                # x: (batch, n) token ids
        mask = Batch.subsequent_mask(x.size(1))          # (1, n, n), True on/below diagonal
        return self.generator(self.blocks(self.embed(x), mask))   # (batch, n, V) log-probs
```

1. What are the inputs? - ids of <s> The ... it is (1, 10)

Our GPT2 - 2019

```
class DecoderOnlyLM(nn.Module):
    def __init__(self, V, N=6, d_model=512, d_ff=1024, h=8, dropout=0.1):
        super().__init__()
        c = copy.deepcopy
        attn, ff = MultiHeadedAttention(h, d_model), PositionwiseFeedForward(d_model, d_ff, dropout)
        self.embed = nn.Sequential(Embeddings(d_model, V), PositionalEncoding(d_model, dropout))
        self.blocks = Encoder(EncoderLayer(d_model, c(attn), c(ff), dropout), N)
        self.generator = Generator(d_model, V)
        [nn.init.xavier_uniform_(p) for p in self.parameters() if p.dim() > 1]
    def forward(self, x):                                # x: (batch, n) token ids
        mask = Batch.subsequent_mask(x.size(1))          # (1, n, n), True on/below diagonal
        return self.generator(self.blocks(self.embed(x), mask))   # (batch, n, V) log-probs
```

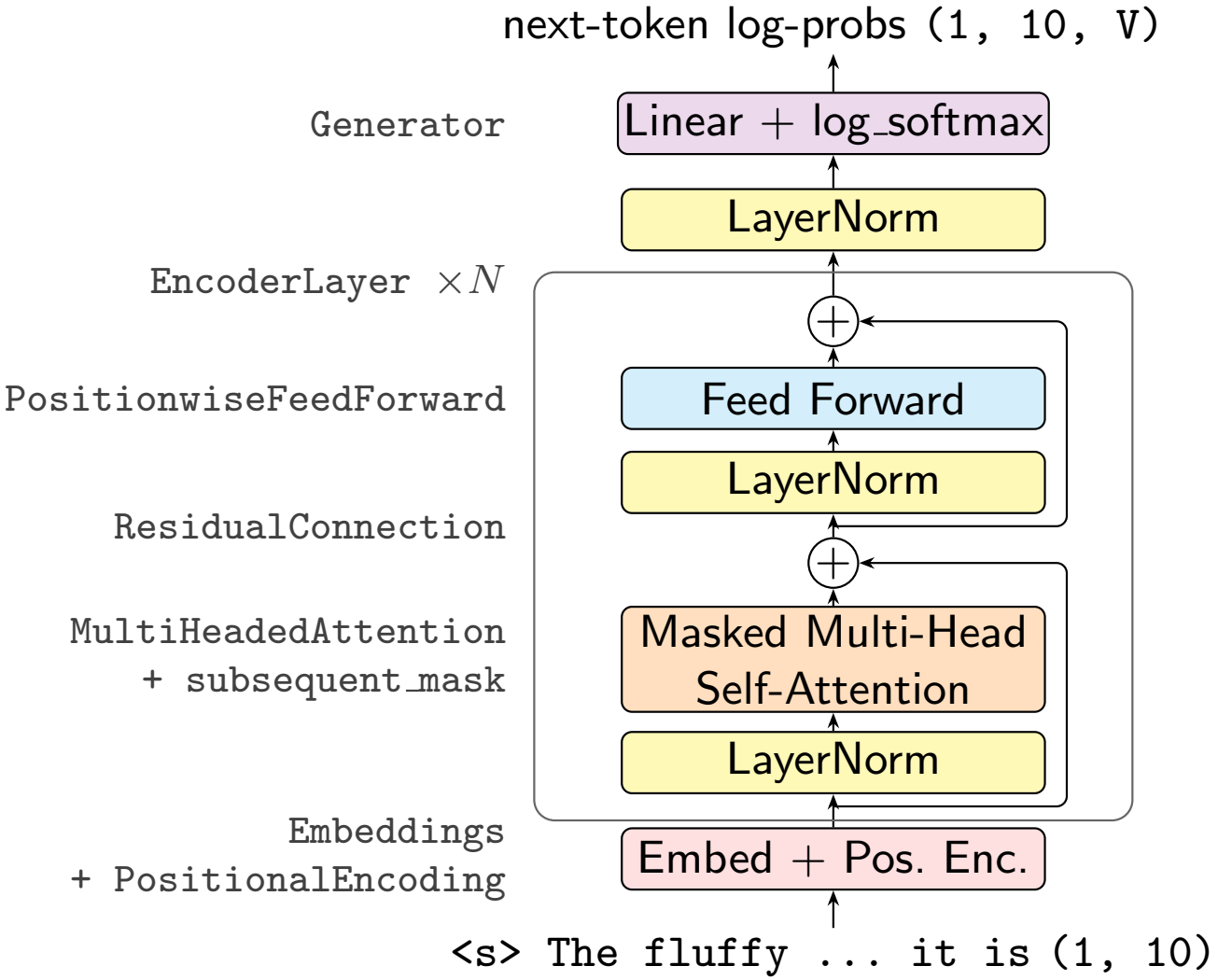
1. What are the inputs? - ids of `<s> The ... it is` `(1, 10)`
2. What are the outputs? - next word log-probs `(1, 10, V)`

Our GPT2 - 2019

```
class DecoderOnlyLM(nn.Module):
    def __init__(self, V, N=6, d_model=512, d_ff=1024, h=8, dropout=0.1):
        super().__init__()
        c = copy.deepcopy
        attn, ff = MultiHeadedAttention(h, d_model), PositionwiseFeedForward(d_model, d_ff, dropout)
        self.embed = nn.Sequential(Embeddings(d_model, V), PositionalEncoding(d_model, dropout))
        self.blocks = Encoder(EncoderLayer(d_model, c(attn), c(ff), dropout), N)
        self.generator = Generator(d_model, V)
        [nn.init.xavier_uniform_(p) for p in self.parameters() if p.dim() > 1]
    def forward(self, x):                                # x: (batch, n) token ids
        mask = Batch.subsequent_mask(x.size(1))          # (1, n, n), True on/below diagonal
        return self.generator(self.blocks(self.embed(x), mask))   # (batch, n, V) log-probs
```

1. What are the inputs? - ids of `<s> The ... it is` `(1, 10)`
2. What are the outputs? - next word log-probs `(1, 10, V)`
3. Called a "decoder-only" model because it performs the decoding step

Decoder-Only Architecture



The Task: Predict the Next Word

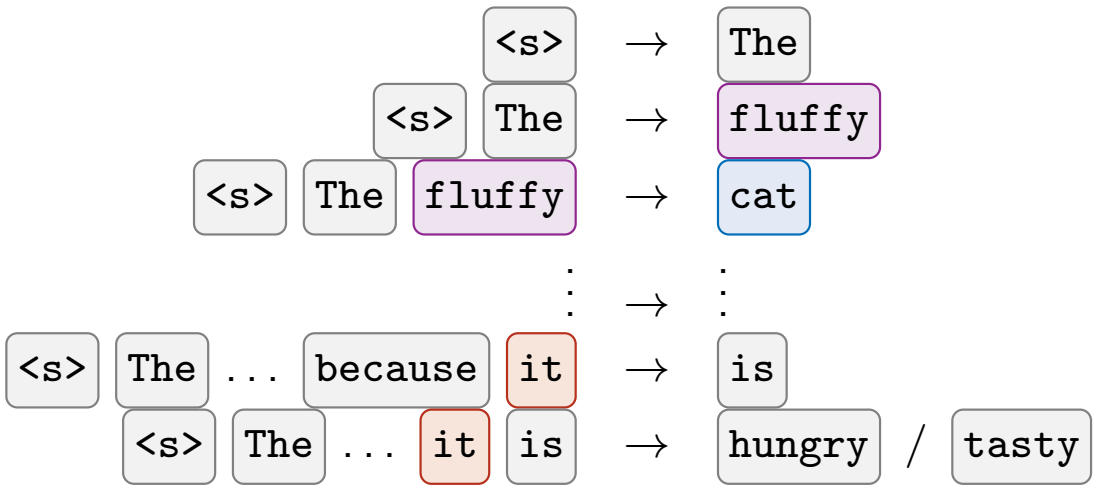
t	0	1	2	3	4	5	6	7	8	9
trg	<s>	The	fluffy	cat	eats	fresh	fish	because	it	is
trg_y	The	fluffy	cat	eats	fresh	fish	because	it	is	hungry

The Task: Predict the Next Word

t	0	1	2	3	4	5	6	7	8	9
trg	<s>	The	fluffy	cat	eats	fresh	fish	because	it	is
trg_y	The	fluffy	cat	eats	fresh	fish	because	it	is	hungry

1. loss function = cross-entropy between soft target `trg_y` and the log-probs from `Generator`

Generating Text - One Word at a Time

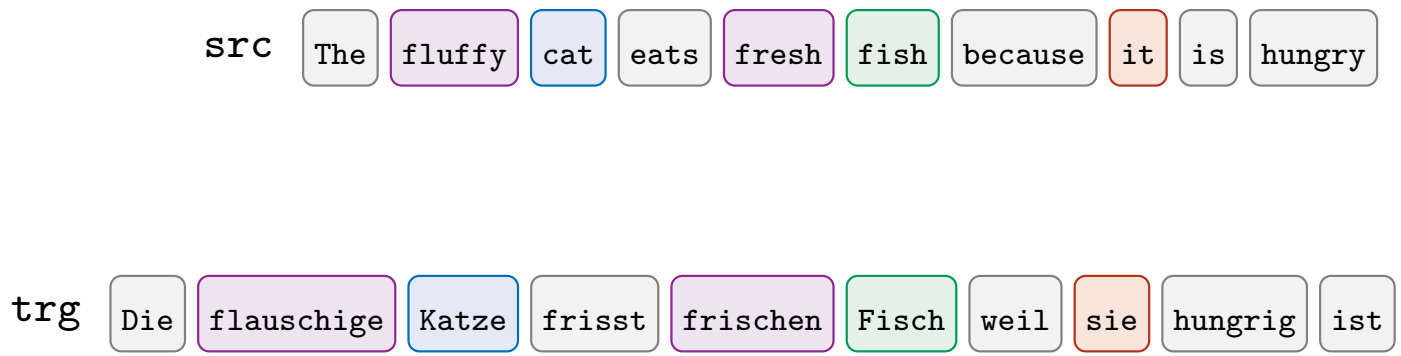


Ok Then What is A Transformer?

1. Imagine you wanted to translate between English and German

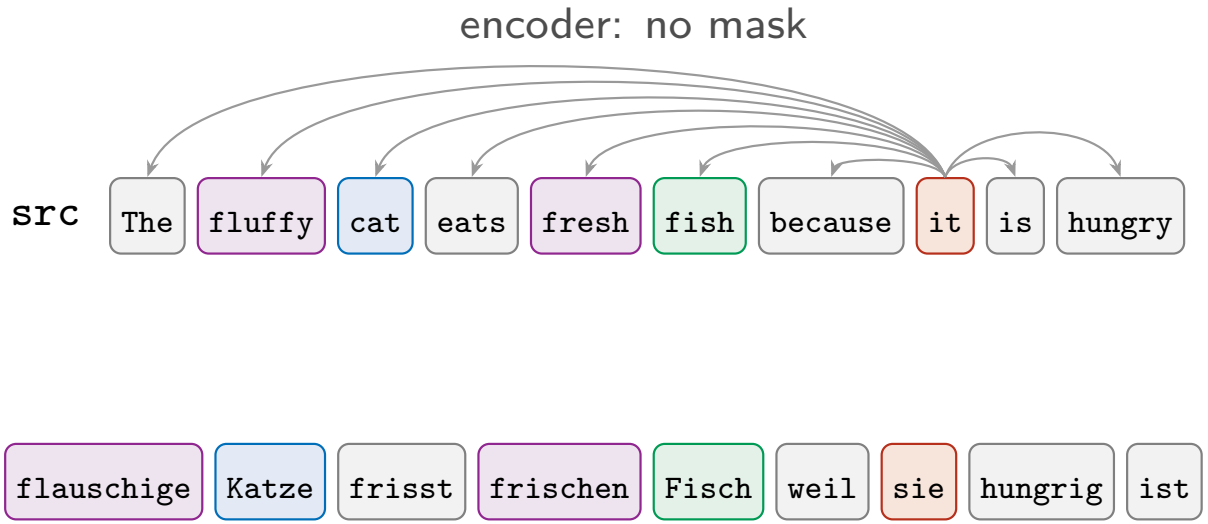
Ok Then What is A Transformer?

1. Imagine you wanted to translate between English and German
2. The *encoder* reads the English (`src`), the *decoder* writes the German (`trg`) one word at a time



Ok Then What is A Transformer?

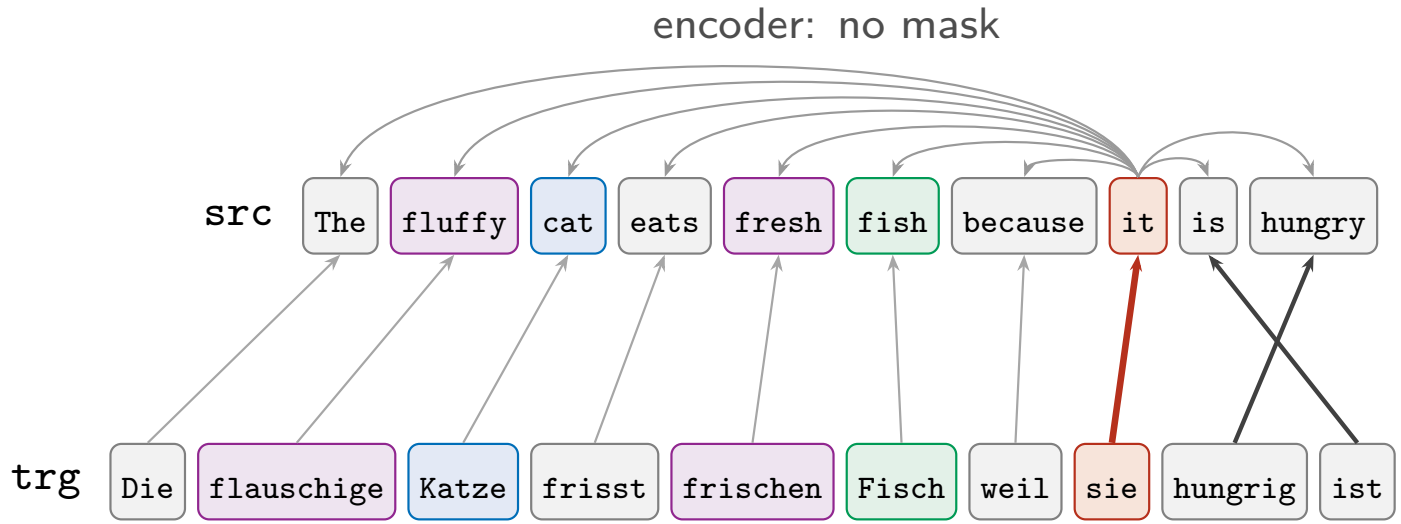
1. Imagine you wanted to translate between English and German
2. The *encoder* reads the English (`src`), the *decoder* writes the German (`trg`) one word at a time



3. The English is all there - drop the causal mask: the *encoder* is our `EncoderLayer` with only a padding `src_mask (1, 1, m)`

Ok Then What is A Transformer?

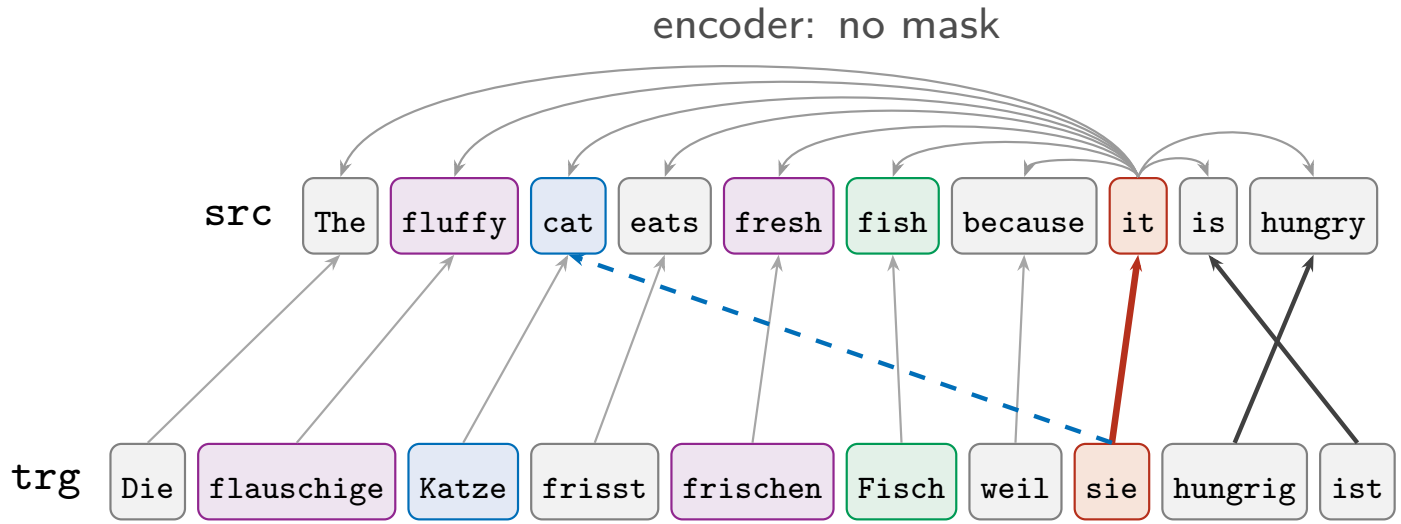
1. Imagine you wanted to translate between English and German
2. The *encoder* reads the English (`src`), the *decoder* writes the German (`trg`) one word at a time



3. The English is all there - drop the causal mask: the *encoder* is our `EncoderLayer` with only a padding `src_mask (1, 1, m)`
4. The decoder still writes left to right, and asks questions *of the English* - *cross-attention* (weights illustrative)

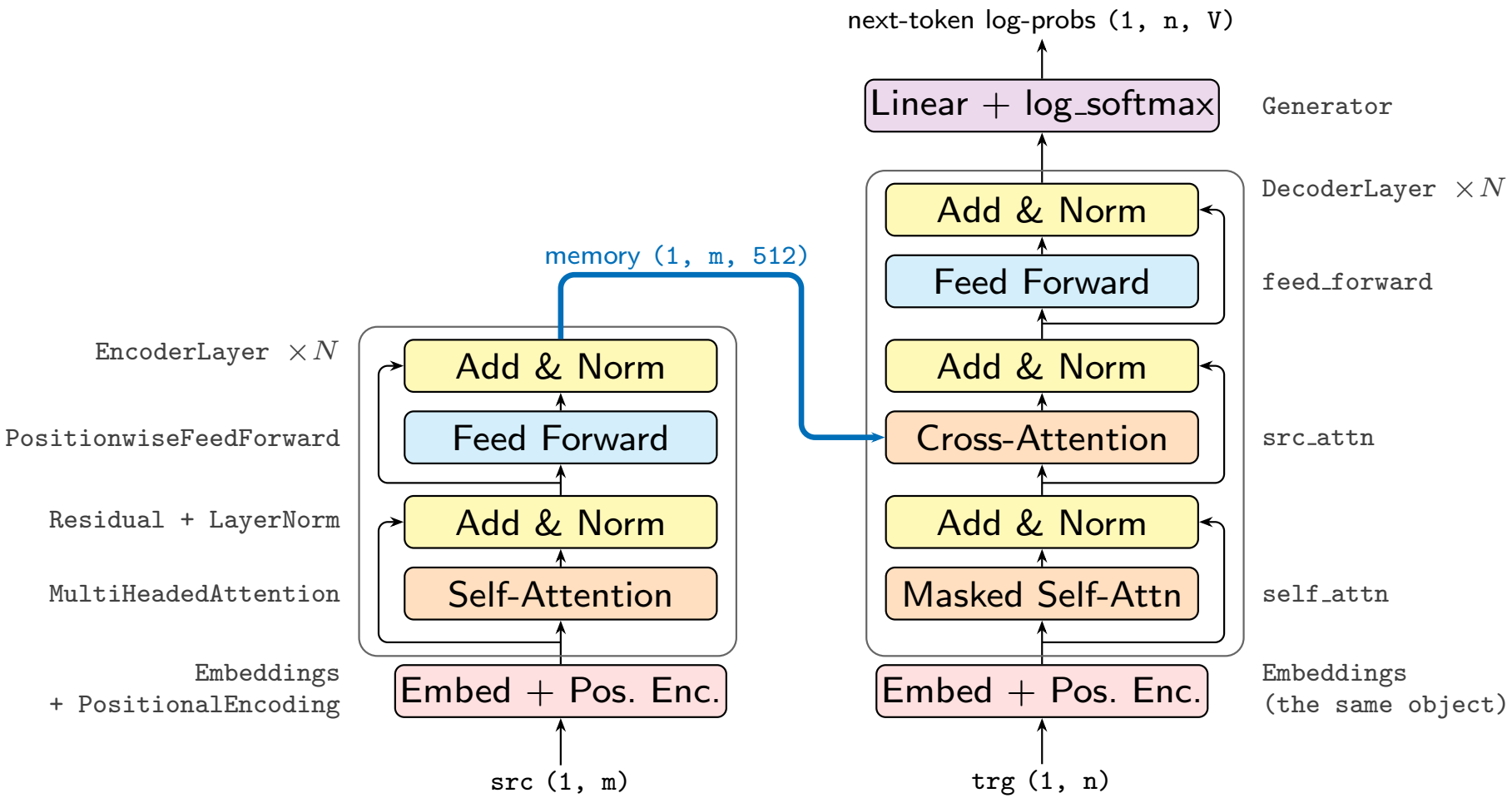
Ok Then What is A Transformer?

1. Imagine you wanted to translate between English and German
2. The *encoder* reads the English (`src`), the *decoder* writes the German (`trg`) one word at a time

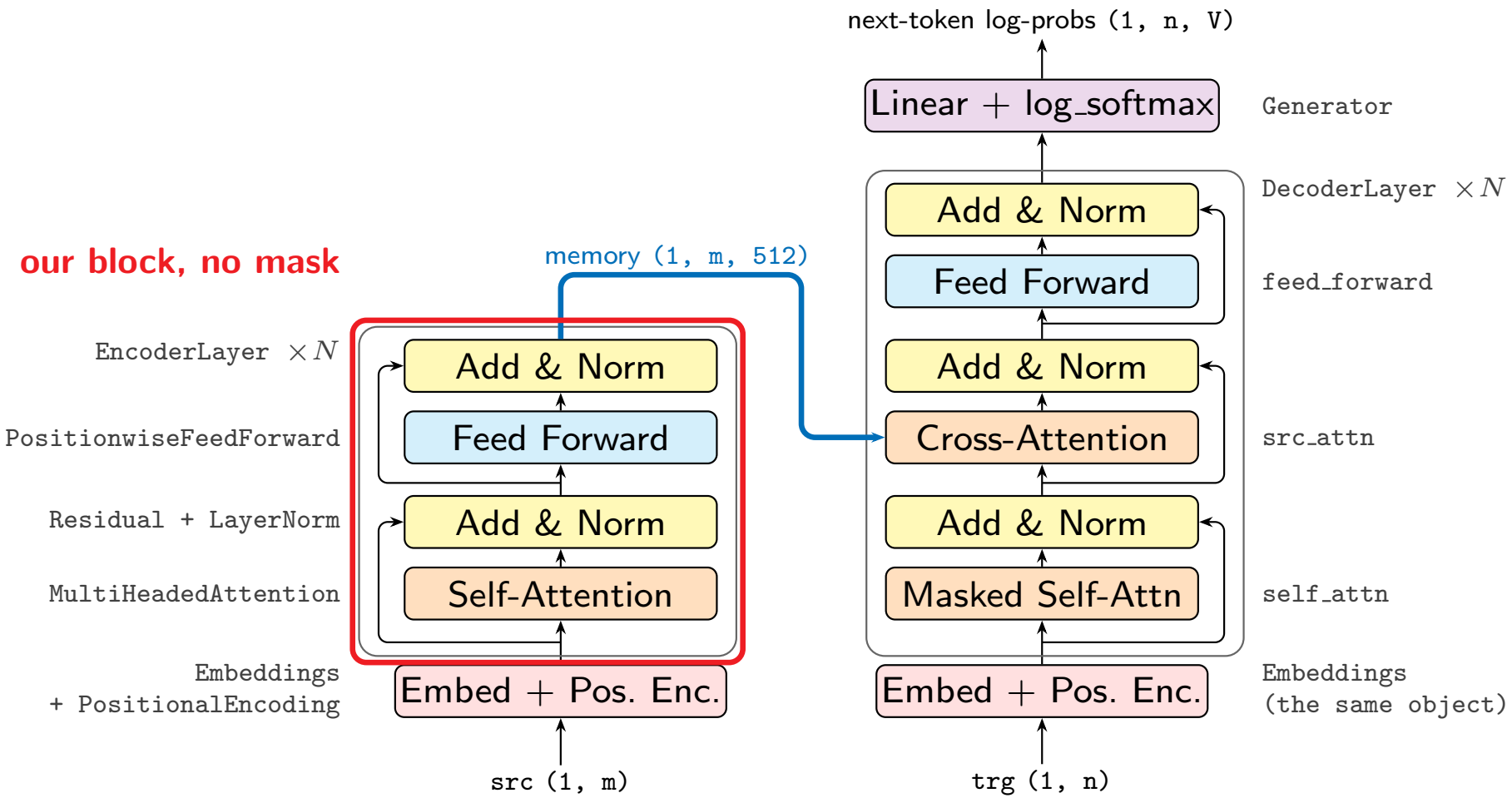


3. The English is all there - drop the causal mask: the *encoder* is our `EncoderLayer` with only a padding `src_mask (1, 1, m)`
4. The decoder still writes left to right, and asks questions *of the English* - *cross-attention* (weights illustrative)
5. `sie`, not `er` - *die* Katze but *der* Fisch, so the decoder needs `it` = `cat`; and `ist` comes last - the lines cross

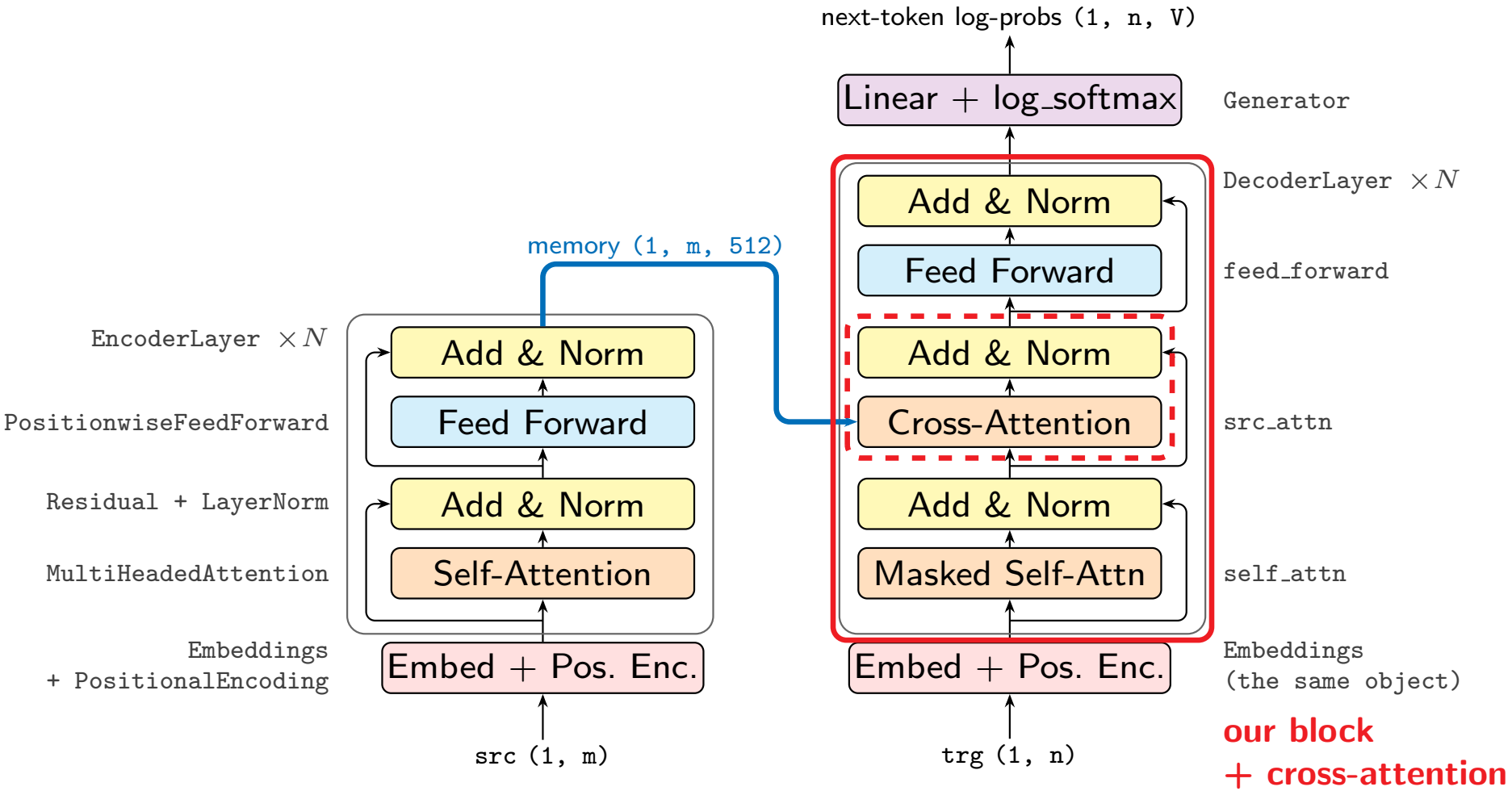
The 2017 Transformer



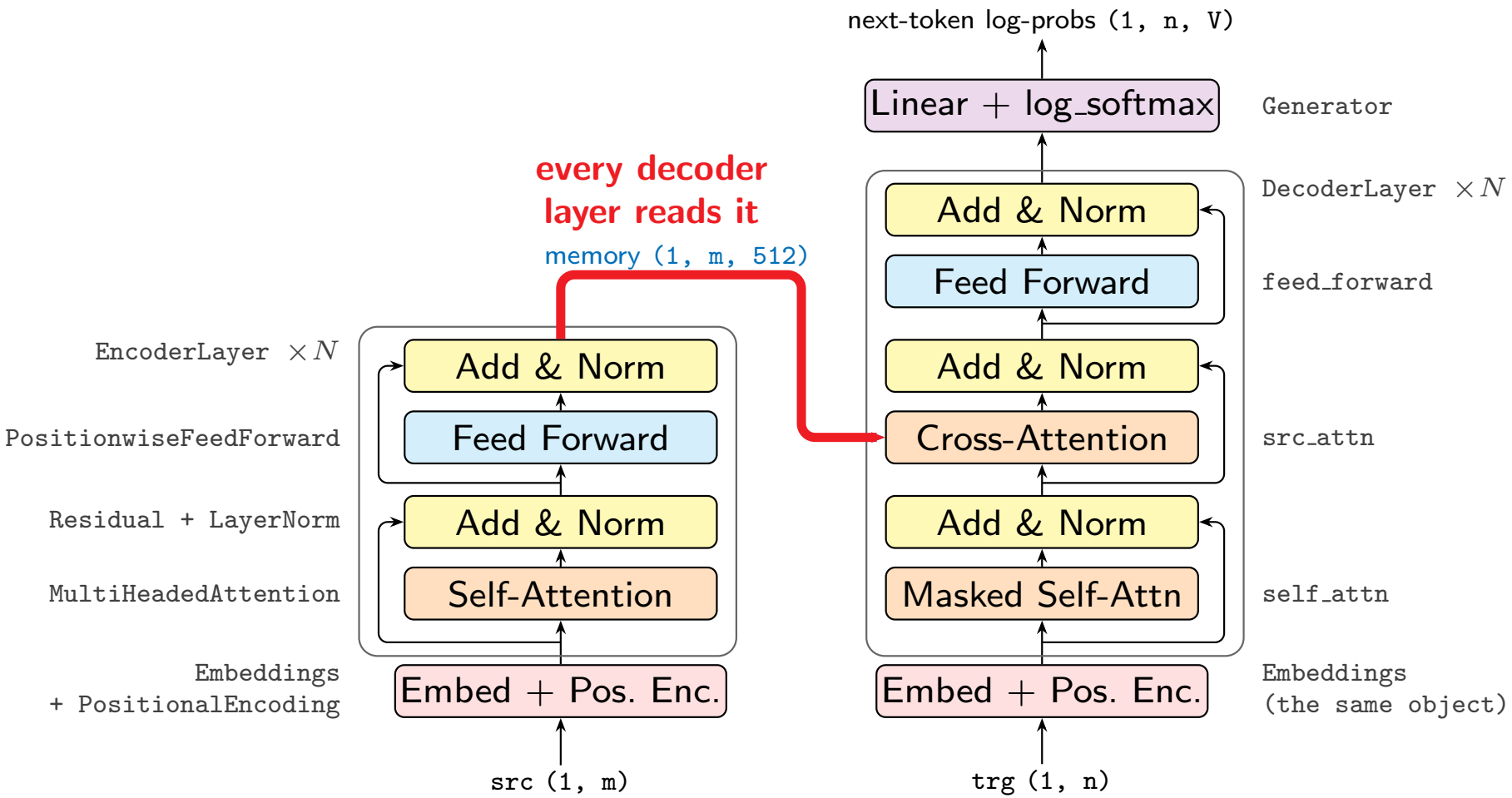
The 2017 Transformer - Encoder



The 2017 Transformer - Decoder



The 2017 Transformer - Memory



Thank you!

Helpful Links

1. Attention Is All You Need - https://arxiv.org/abs/1706.03762
2. The Annotated Transformer - https://nlp.seas.harvard.edu/2018/04/03/attention.html
3. Code - https://github.com/GersiD/ML/tree/main/transformer
4. Slides - Email me Gersi.Doko@unh.edu